

FCMP++ Function Analysis

*A comprehensive mathematical and code based
review of functions for use in FCMP.*

Joshua Babb
Brandon Goodell
Rigo Salazar
Freeman Slaughter
Luke SZramowski





Contents

1	Structure	6
2	Constant Time	7
3	Faster Transparent Amount Commitments	8
3.1	Mathematical Overview	8
3.2	PR #10108 - H_Table	9
3.2.1	Functionality	9
3.2.2	Implementation Observations	10
3.2.3	Edge Case Analysis	11
3.2.4	Fuzzing Results	11
3.3	PR #10108 - zeroCommitVartime	12
3.3.1	Functionality	12
3.3.2	Implementation Observations	13
3.3.3	Edge Case Analysis	13
3.3.4	Fuzzing Results	14
3.4	Section Conclusion	14
4	Batch Inverse	15
4.1	Mathematical Overview	15
4.2	PR #10111 - fe_equals	16
4.2.1	Functionality	16
4.2.2	Implementation Observations	16
4.2.3	Edge Case Analysis	17
4.2.4	Fuzzing Results	17
4.3	PR #10111 - fe_batch_invert	18
4.3.1	Functionality	18
4.3.2	Implementation Observations	19
4.3.3	Edge Case Analysis	20
4.3.4	Fuzzing Results	20
4.4	PR #10111 - fe_frombytes_vartime	21
4.4.1	Functionality	21
4.4.2	Implementation Observations	23
4.4.3	Edge Case Analysis	24
4.4.4	Fuzzing Results	24



4.5	Section Conclusion	25
5	ED25519 to X25519 Point Conversion	26
5.1	Mathematical Overview	26
5.2	PR #9828 - edwardsYZ_to_x25519	27
5.2.1	Functionality	27
5.2.2	Implementation Observations	27
5.2.3	Edge Case Analysis	28
5.2.4	Fuzzing Results	28
5.3	PR #9828 - ge_p3_to_x25519	29
5.3.1	Functionality	29
5.3.2	Implementation Observations	29
5.3.3	Fuzzing Results	29
5.4	PR #9828 - edwards_bytes_to_x25519_vartime	30
5.4.1	Functionality	30
5.4.2	Implementation Observations	30
5.4.3	Edge Case Analysis	31
5.4.4	Fuzzing Results	31
5.5	Section Conclusion	32
6	Torsion Clearing	33
6.1	Mathematical Overview	33
6.2	PR #10342 - mul8_is_identity	36
6.2.1	Functionality	36
6.2.2	Implementation Observations	36
6.2.3	Edge Case Analysis	37
6.2.4	Fuzzing Results	37
6.3	PR #10342 - clear_torsion	38
6.3.1	Functionality	38
6.3.2	Implementation Observations	38
6.3.3	Edge Case Analysis	39
6.3.4	Fuzzing Results	39
6.4	PR #10342 - get_valid_cleared_torsion_point	40
6.4.1	Functionality	40
6.4.2	Implementation Observations	40
6.4.3	Edge Case Analysis	41



6.4.4	Fuzzing Results	41
6.5	Section Conclusion	42
7	Change to fe_reduce_vartime	43
7.1	PR #10135 - fe_reduce_vartime	43
7.1.1	Functionality	43
7.1.2	Implementation Observations	43
7.1.3	Edge Case Analysis	44
7.1.4	Fuzzing Results	44
7.2	Section Conclusion	45
8	ED25519 to Weierstrass Point Conversion	46
8.1	Mathematical Overview	46
8.2	PR #10345 - fe_ed_derivatives_to_weix	47
8.2.1	Functionality	47
8.2.2	Implementation Observations	47
8.3	PR #10345 - fe_ed_derivatives_to_weix_y	48
8.3.1	Functionality	48
8.3.2	Implementation Observations	48
8.3.3	Edge Case Analysis	49
8.3.4	Fuzzing Results	49
8.4	PR #10345 - point_to_ed_derivatives	50
8.4.1	Functionality	50
8.4.2	Implementation Observations	51
8.4.3	Fuzzing Results	51
8.5	Section Conclusion	51
9	Key Image	52
9.1	Mathematical Overview	52
9.2	PR #10338 - biased_hash_to_ec	53
9.2.1	Functionality	53
9.2.2	Implementation Observations	53
9.2.3	Fuzzing Results	54
9.3	PR #10338 - unbiased_hash_to_ec	55
9.3.1	Functionality	55
9.3.2	Proof of Unbiasedness	57
9.3.3	Implementation Observations	59



9.3.4	Edge Case Analysis	60
9.3.5	Fuzzing Results	60
9.4	PR #10338 - derive_key_image_generator	61
9.4.1	Functionality	61
9.4.2	Implementation Observations	61
9.5	PR #10338 - generate_key_image	62
9.5.1	Functionality	62
9.5.2	Implementation Observations	62
9.5.3	Fuzzing Results	62
9.6	Section Conclusion	63
10	Converting Outputs to Tuples	64
10.1	Mathematical Overview	64
10.2	PR #10360 - output_to_tuple	64
10.2.1	Functionality	64
10.2.2	Implementation Observations	67
10.2.3	Edge Case Analysis	67
10.2.4	Fuzzing Results	67
10.3	PR #10360 - output_tuple_to_pre_leaf_tuple	68
10.3.1	Functionality	68
10.3.2	Implementation Observations	69
10.3.3	Fuzzing Results	69
10.4	Section Conclusion	70
11	Constructing the Selene Scalar	71
11.1	PR #10359 - selene_scalar_from_bytes	71
11.1.1	Functionality	71
11.1.2	Implementation Observations	71
11.1.3	Edge Case Analysis	72
11.1.4	Fuzzing Results	72
11.1.5	Conclusion	72
12	ED25519 to Curve Tree	74
12.1	PR #10360 - output_to_leaves	74
12.1.1	Functionality	74
12.1.2	Implementation Observations	81
12.1.3	Edge Case Analysis	81



12.1.4 Fuzzing Results	81
12.1.5 Conclusion	82
13 Conclusion	83



1 Structure

This document is structured with each section addressing a particular point in the audit request. Every section is structured similarly: we begin with a mathematical overview of the purpose of the code covered in each section, proceed with an analysis of all pertinent functions, then end with a section that addresses the questions included in the audit request.

Each instance of a function's analysis is structured identically:

1. **Functionality:** This segment presents the code for the analyzed functions as written in the repository, along with a Source-to-Specification map detailing the operations contained within the function. Additionally, it includes where the code can be found in the repository.
2. **Implementation Observations:** In this component, we include our observations surrounding the code, how it implements the underlying mathematical concepts, and any potential issues that may arise from a disconnect between the code and the math.
3. **Edge Case Analysis:** Here, we deal with determining how the code handles extreme, rare, or impossible inputs. The purpose is mainly to determine if there are fringe cases which the function cannot handle, but additionally this analysis provides insight into potential guardrails that may need to be placed on the function.
4. **Fuzzing Results:** Fuzzing is a compilation of results wherein the function is executed on a variety of platforms and infrastructure over a wide range of inputs, with the goal of determining if there are any purely code-based concerns to be addressed.
5. **Conclusion:** Finally, the section conclusion will be a short determination of our professional opinions regarding if the code is fit for deployment or not. For ease of reading, a green checkmark ✓, yellow checkmark ⚠, or red x ✗ will be included to indicate, respectively: yes, yes with minor concerns, or no.

If there are any additional results, we include them in a dedicated section with the relevant function. Additionally, if there is no edge case analysis section, then it should be interpreted that all relevant behavior is captured in the fuzzing results or that there were no notable interactions.

As a convention, when code is being discussed, the first two columns we label L1 and L2. Firstly, L1 is the line location in the source code, while L2 is the code's line location in the document.



2 Constant Time

We note that constant-timedness is critical for the implementation of various functions utilized in FCMP++. The purpose is to ensure that timing information is not leaked during the execution of the function: as all computations run using the same amount of time regardless of input, timing analysis loses its utility. These considerations are only necessary in the case that the function inputs are private; indeed if all the information needed to initialize the function is public, then no advantage can be gained from a timing attack, so constant time is not necessary. With this in mind, the function is free to default to whichever method results in the fastest runtime.



3 Faster Transparent Amount Commitments

In this section we seek to analyze the components of the `zeroCommitVartime` process. Our focus is to determine the following:

1. Is the cryptography implementation sound?
2. Is the cryptography implemented correctly?

The first subsection will deal with the cryptographic ideas that are used in the analyzed functions, while the following subsections will answer the two questions above for the functions under consideration.

3.1 Mathematical Overview

Definition 3.1: Pedersen Commitments

A **Pedersen commitment** C to an amount a with blinding factor r is defined as:

$$C = r \cdot G + a \cdot H$$

where G, H are both elliptic curve points such that the discrete logarithm relationship between them is unknown and presumed to be difficult to determine.

A **zero commitment** to an amount a is defined as a Pedersen Commitment with a blinding factor equal to 1:

$$C_{zero} = 1 \cdot G + a \cdot H$$

For the purpose of this PR, G is the base point for ED25519 and H is Monero's second generator, which is a hash of G .

Definition 3.2: Binary Decomposition

Any 64 bit unsigned integer a can be uniquely represented as:

$$a = \sum_{k=0}^{63} a_k \cdot 2^k$$

where $a_k \in \{0, 1\}$.

There are a number of different point representations that are used in the code, which are used for the sake of efficiency. In general, a point (x, y) on a *Twisted Edwards Curve*



given by $-x^2 + y^2 = 1 + dx^2y^2$ can be represented in the following ways. For ED25519, the curve parameter is the constant $d = -\frac{121665}{121666} \pmod{p}$, where $p = 2^{255} - 19$.

Function	Coordinates	Use
ge_p3	$(X : Y : Z : T)$ where $x = X/Z, y = Y/Z, T = (XY)/Z$	Extended Twisted Edwards Coordinates
ge_cached	$(Y + X : Y - X : Z : 2dT)$	Optimized for Point Addition for ge_add
ge_p1p1	$(X : Y : Z : T)$ where $x = X/Z, y = Y/T$	Output of ge_add and ge_p3_dbl Needs to be converted.

3.2 PR #10108 - H_Table

3.2.1 Functionality

The `H_Table` function returns a static table of 64 precomputed points of the form $2^i \cdot H$, in the `ge_cached` coordinate form. Explicitly:

$$\text{H_Table}[i] = 2^i \cdot H$$

Listing 1: h_table Code

```

1 static constexpr std::size_t H_TABLE_SIZE = 64;
2 const std::vector<ge_cached>& H_TABLE()
3 {
4     struct static_h_table
5     {
6         std::vector<ge_cached> h_table;
7
8         static_h_table()
9             : h_table()
10        {
11            h_table.resize(H_TABLE_SIZE);
12            ge_p3_to_cached(&h_table.at(0), &ge_p3_H);
13            ge_p3 H_bit_p3 = ge_p3_H;
14
15            for (std::size_t i = 1; i < H_TABLE_SIZE; ++i)
16            {
17                ge_p1p1 H_bit_p1p1;
18                ge_p3_dbl(&H_bit_p1p1, &H_bit_p3);

```



```

19         ge_p1p1_to_p3(&H_bit_p3, &H_bit_p1p1);
20         ge_p3_to_cached(&h_table.at(i), &H_bit_p3);
21     }
22 }
23 };
24 static const static_h_table out;
25 return out.h_table;
26 }

```

Table 1: H_Table Source To Spec Map

L1	L2	Code	Purpose
220	5	H_TABLE_SIZE = 64	One entry per bit of uint64_t
221	6	const std::vector<ge_cached>& H_TABLE()	Returns table of $2^i \cdot H$, $i = 0 \dots 63$
229	11	h_table.resize(H_TABLE_SIZE)	Allocate 64 entries.
231	12	ge_p3_to_cached(&h_table.at(0), &ge_p3_H)	Sets H_TABLE[0] = H.
		<i>Converts from ge_p3 to ge_cached form.</i>	
232	13	ge_p3 H_bit_p3 = ge_p3_H	Sets working var = H
234	15	Doubling Loop	H_TABLE[i] = $2 \cdot H_TABLE[i-1]$
237	18	ge_p3_dbl(&H_bit_p1p1, &H_bit_p3)	Doubling. <i>Output is ge_p1p1</i>
			<i>(intermediate representation)</i>
238	19	ge_p1p1_to_p3(&H_bit_p3, &H_bit_p1p1)	Convert intermediate to
			extended for next iteration.
239	20	ge_p3_to_cached(&h_table.at(i), &H_bit_p3)	Convert from extended to cached
			for storage.
243	24	Meyers singleton	Thread-safe in C++11.
			Constructed once on first call

Located in rctOps.cpp

3.2.2 Implementation Observations

1. We note that the code utilizes all three representations presented here when running. This is since `ge_add` requires `ge_p3` and `ge_cached` to produce `ge_p1p1`. The table



stores `ge_cached` *specifically* to avoid the per iteration conversion. The original spec simply says ‘add points’.

3.2.3 Edge Case Analysis

`H_TABLE` is a parameter-less Meyers singleton, so it has no input-driven edge cases of its own. The table below records the structural invariants exercised on first call:

Invariant	Behavior
First call	Allocates 64 entries, computes <code>H_TABLE[0..63]</code> via the doubling loop, stores in <code>ge_cached</code> form.
Subsequent calls	Returns the same <code>const std::vector<ge_cached>&</code> reference (no recomputation; thread-safe under C++11).
<code>H_TABLE[0]</code>	Equals <code>ge_p3_H</code> converted to <code>ge_cached</code> .
<code>H_TABLE[i]</code> , $1 \leq i \leq 63$	Equals $2 \cdot H_TABLE[i - 1] = 2^i \cdot H$, computed via <code>ge_p3_dbl</code> then converted.

3.2.4 Fuzzing Results

`H_TABLE` is exercised indirectly by every `zeroCommitVartime` fuzz iteration: the AFL++ harness `afl_zero_commit` ran 101,096,575 random `uint64_t` amounts on the PR-10108 branch (600s wall, 18,494 exec/s, ASan/UBSan), with zero memory-safety findings and zero invariant violations. Because the table is built once on first call and read-only thereafter, the campaign primarily certifies correct singleton initialization across the full 64-bit amount space.



3.3 PR #10108 - zeroCommitVartime

3.3.1 Functionality

Listing 2: zeroCommitVartime Code

```
1 key zeroCommitVartime(xmr_amount amount) {
2     const zero_commitment* begin = zero_commitments;
3     const zero_commitment* end = zero_commitments + sizeof(
4         zero_commitments)
5         / sizeof(zero_commitments[0]);
6     const zero_commitment value{ amount, rct::zero() };
7     const auto it = std::lower_bound(begin, end, value,
8         [](const zero_commitment& e0, const zero_commitment& e1)
9         { return e0.amount < e1.amount; });
10    if (it != end && it->amount == amount)
11    {
12        return it->commitment;
13    }
14
15    ge_p3 res_ge_p3 = get_G_p3();
16
17    static_assert(sizeof(xmr_amount) * 8 == H_TABLE_SIZE,
18        "unexpected size of h table");
19
20    for (size_t i = 0; i < H_TABLE_SIZE; ++i)
21    {
22        if (amount & (xmr_amount(1) << i))
23        {
24            ge_p1p1 p1p1;
25            ge_add(&p1p1, &res_ge_p3, &H_TABLE()[i]);
26            ge_p1p1_to_p3(&res_ge_p3, &p1p1);
27        }
28    }
29    rct::key res;
30    ge_p3_tobytes(res.bytes, &res_ge_p3);
31    return res;
32 }
```



Table 2: zeroCommitVartime Source To Spec Map

L1	L2	Code	Purpose
349	1	key zeroCommitVartime(xmr_amount amount)	Computes $C = G + a \cdot H$
350		Binary search in zero_commitments[]	Common Value Search.
358	14	ge_p3 res_ge_p3 = get_G_p3()	Sets $C = G$
359	2	static_assert(...)	Compile-time guard:
			amount width == table size
360	13	for (size_t i = 0; i < H_TABLE_SIZE; ++i)	Iterate all 64 bits
362	15	if (amount & (xmr_amount(1) << i))	Test bit i .
365	18	ge_add(&p1p1, &res_ge_p3, &H_TABLE()[i])	$C += 2^i \cdot H$ when bit i is set.
366	19	ge_p1p1_to_p3(&res_ge_p3, &p1p1)	Convert result back to ge_p3 for next addition.
370	20	ge_p3_tobytes(res.bytes, &res_ge_p3)	Compress to 32-byte Edwards
			encoding.

Located in rctOps.cpp

3.3.2 Implementation Observations

1. In lines 44 – 218, there is a hardcoded, sorted array of amount/commitment pairs for common Monero denominations, which are binary searched before `H_Table` runs. These values are constant and are not derived at runtime.

3.3.3 Edge Case Analysis

Input	Behavior
Amount = 0	Cache hit. Returns G.
Amount = 2^{32}	First amount that triggered the 1UL bug. Now works correctly.
Amount = $2^{64} - 1$	All 64 table entries added. Maximum execution time.
Amount = 2^{63}	Uses <code>H_TABLE[63]</code> only. Tests the last table entry.



3.3.4 Fuzzing Results

The AFL++ harness `afl_zero_commit` fuzzed `zeroCommitVartime`, completing 101,096,575 iterations. For every random `uint64_t` amount the harness verified determinism (same input $\Rightarrow$ same output) and that the output decompresses as a valid Ed25519 point via `ge_frombytes_vartime`. No crashes, memory-safety findings, or invariant violations were found. The 1UL bug class is exercised by an adversarial seed corpus that includes $a = 2^{32}$ and other powers-of-two boundaries.

3.4 Section Conclusion



All functions used for the *Faster Transparent Amount Commitments* are described correctly mathematically, and implemented according to that description. As such, it is our conclusion that the above functions are fit for deployment in Monero.



4 Batch Inverse

4.1 Mathematical Overview

The following code performs the Montgomery trick for group inversion. Instead of inverting n group elements individually, it batches them together and inverts once, which still results in the inversions of each of the elements, but requires 1 inversion and $3(n - 1)$ group multiplications as opposed to n costly inversions.

The trick works as follows: let $a_1, \dots, a_n$ be the group elements one wishes to invert. Perform the following:

1. For each $1 \leq i \leq n$, compute the product:

$$P_i = \prod_{k=1}^i a_k$$

2. Invert the product of all the elements:

$$I_n = P_n^{-1}$$

3. For each $1 \leq i \leq n$, compute the following:

$$I_{i-1} \equiv I_i \cdot a_i \pmod{p}$$

$$a_i^{-1} \equiv P_{i-1} \cdot I_i \pmod{p}$$

We can see that, since there are n elements, Line 1 requires $n - 1$ multiplications (since $P_1 = a_1$). Line 2 is the expensive operation, requiring 1 inversion. For the first part of Line 3, we require $n - 1$ multiplications, since it terminates for $i = 2$. Similarly, we require $n - 1$ multiplications for the second part, bringing the total operation count to $3(n - 1)$ multiplications and 1 inversion.



4.2 PR #10111 - fe_equals

4.2.1 Functionality

Listing 3: fe_equals Code

```
1 int fe_equals(const fe a, const fe b) {
2     unsigned char a_bytes[32];
3     unsigned char b_bytes[32];
4     fe_tobytes(a_bytes, a);
5     fe_tobytes(b_bytes, b);
6
7     int r = 1;
8     for (int i = 0; i < 32; ++i) {
9         r &= a_bytes[i] == b_bytes[i];
10    }
11    return r;
12 }
```

Table 3: fe_equals Source To Spec Map

L1	L2	Code	Purpose
320	4	fe_tobytes(a_bytes, a)	a_bytes is the canonical 32 byte LE encoding.
321	5	fe_tobytes(b_bytes, b)	b_bytes is the canonical 32 byte LE encoding.
323	7	int r = 1	Initial Assumption: Assume Equality
324-325	9	r &= a_bytes[i] == b_bytes[i]	$r = \prod_{i=0}^{31} (a_bytes[i] \wedge b_bytes[i])$
327	11	return r	1 iff $a = b$, 0 otherwise

Located in crypto-ops.c

4.2.2 Implementation Observations

1. The constant-timedness might rely on compiler cooperation. The loop on Line 324-325 has no early exit in the code, but the compiler might terminate when $r == 0$ for the first time.
2. If fe_tobytes is variable time, then fe_equal will also be variable time.



3. The AND-accumulator pattern (`r &= a==b`) is theoretically vulnerable to as-if-rule short-circuit at higher optimization levels: once `r` becomes 0, the loop result is fixed and a sufficiently aggressive compiler could exit early. An XOR-accumulator (`r |= a[i] ^ b[i]`) is provably immune since every iteration contributes data-dependently, and would be a low-cost defense-in-depth substitution. Constant-time has been verified in the current build at `-O2` under Clang 18 via Timecop, so this is forward-looking hardening rather than a present leak.

4.2.3 Edge Case Analysis

Input	Behavior
Same Pointer <code>a == b</code>	Returns 1 (serializes both identically)
Different Limb Rep. for Same Value	Returns 1, <code>fe_tobytes</code> canonicalizes the value
$a = 0, b = p$	Returns 1, p reduces to zero.

4.2.4 Fuzzing Results

`fe_equals` does not have its own dedicated fuzz harness. Instead, it is exercised indirectly inside `afl_batch_invert`, where it is the comparator that checks the batch result against the per-element `fe_invert` oracle. Across the 905,249 iterations of that campaign it was therefore executed on every input drawn from both the canonical and the near-prime corpus. A constant-time tripwire `ct_fe_equals` runs the loop body under Valgrind/Memcheck on poisoned inputs and reports no branch-on-secret leaks at `-O2`. No mismatches between `fe_equals` and the limb-level equality oracle were observed across any campaign.



4.3 PR #10111 - fe_batch_invert

4.3.1 Functionality

Listing 4: fe_batch_inverse Code

```
1 void fe_batch_invert(fe* out, const fe* in, const int n) {
2     if (n == 0) {
3         return;
4     }
5
6     assert(out);
7     assert(in);
8     #ifndef NDEBUG
9     {
10         const fe* pmin = out < in ? out : in;
11         const fe* pmax = out > in ? out : in;
12         assert(pmin + n < pmax);
13     }
14 #endif
15     fe_copy(out[0], in[0]);
16
17     for (int i = 1; i < n; ++i) {
18         fe_mul(out[i], out[i - 1], in[i]);
19     }
20     fe a;
21     fe_invert(a, out[n - 1]);
22
23     for (int i = n; i > 1; --i) {
24         fe_mul(out[i - 1], a, out[i - 2]);
25         fe_mul(a, a, in[i - 1]);
26     }
27     fe_copy(out[0], a);
28 }
```

Table 4: fe_batch_inverse Source To Spec Map

L1	L2	Code	Purpose
333-335	2	<code>if (n == 0) return</code>	Empty Batch No-Op
337-338	6-7	<code>assert(out); assert(in)</code>	Null Pointer Guard
339-346	10-11	<code>pmin+n < pmax</code>	No Overlap Check
349	15	<code>fe_copy(out[0], in[0])</code>	$P_1 = f_1$
350-352	17-19	<code>fe_mul(out[i], out[i-1], in[i])</code>	$P_i = P_{i-1} \cdot f_i$
356	21	<code>fe_invert(a, out[n-1])</code>	$Q = P_n^{-1}$
360	24	<code>fe_mul(out[i-1], a, out[i-2])</code>	$f_i^{-1} = Q \cdot P_{i-1}$
361	25	<code>fe_mul(a, a, in[i-1])</code>	$Q \leftarrow Q \cdot f_i$
363	27	<code>fe_copy(out[0], a)</code>	$f_1^{-1} = Q$

Located in crypto-ops.c

4.3.2 Implementation Observations

1. `out[]` is dual-use: the forward pass stores the prefix products in `out[i]`, the backward pass reads `out[i-2]` and overwrites `out[i-1]`. This is done so that none of the prefix products are consumed after an overwrite.
2. There is no zero-input guard. If any `in[i] == 0`, then the prefix product becomes zero from that point onward. The inversion step returns zero after leveraging Fermat's Little Theorem, in essence sterilizing the output results. To prevent this fruit from the poisonous tree, non-zero inputs must be guaranteed when called.
3. There is also no guard in place to prevent negative n . Since a negative value for n still passes the `n == 0` check, the function proceeds to `fe_invert(a, out[n-1])` with a negative index, reading out of bounds: this is the actual source of undefined behavior, not the (then-skipped) loops. Practically, this will never happen, but it should be noted regardless.
4. The overlap assertion is debug-only, so release builds will have no aliasing protect. Aliasing corrupts output silently.



-
5. There is a potential bug in the header docstring. The document comment says `@return: 0 on success`, but the function signature is `void`. No return value exists.

4.3.3 Edge Case Analysis

Input	Behavior
$n = 0$	Early return, no memory access.
$n = 1$	Degenerates to a single <code>fe_invert</code> .
$n < 0$	Undefined Behavior: reads <code>out[n-1]</code> with a negative index.
Any $in[i] = 0$	Silent Corruption: All outputs are wrong.
Out and In Overlap	Undefined Behavior in Release; Assertion in Debug.
<code>out == NULL</code> or <code>in == NULL</code> ($n > 0$)	Undefined Behavior in Release; Assertion in Debug.

4.3.4 Fuzzing Results

The AFL++ harness `afl_batch_invert` ran 905,249 total iterations across the ASan/UBSan (151,475), CMPLOG (49,852), MSan (675,901), and extended-ASan (29,021) rounds. ASan reported no heap overflows, use-after-frees, or buffer overruns, and MSan reported no uninitialized-memory reads inside the batch routine itself. Across the rounds AFL++ recorded 148 SIGILL property violations from the harness’s `__builtin_trap()` assertion (80 CMPLOG + 58 MSan), all triaged to a single root cause: inputs containing zero or near-zero field elements. This is the documented precondition violation discussed in observation 2 above: a single zero in the prefix product propagates through the inversion via `fe_invert`’s Fermat-style identity, corrupting every output. Protocol-level callers exclude zero inputs by construction, so the violations do not require escalation.



4.4 PR #10111 - fe_frombytes_vartime

4.4.1 Functionality

Listing 5: fe_frombytes_vartime Code

```
1 int fe_frombytes_vartime(fe y, const unsigned char* s) {
2     int64_t h0 = load_4(s);
3     // Unpack bits [0,31] into limb 0
4     int64_t h1 = load_3(s + 4) << 6;
5     // Unpack bits [32,55], shift for limb 1 position
6     int64_t h2 = load_3(s + 7) << 5;
7
8     int64_t h3 = load_3(s + 10) << 3;
9
10    int64_t h4 = load_3(s + 13) << 2;
11
12    int64_t h5 = load_4(s + 16);
13
14    int64_t h6 = load_3(s + 20) << 7;
15
16    int64_t h7 = load_3(s + 23) << 5;
17
18    int64_t h8 = load_3(s + 26) << 4;
19
20    int64_t h9 = (load_3(s + 29) & 8388607) << 2;
21    // Top limb: mask 0x7FFFFFF clears bit 255 (Ed25519 sign bit)
22
23    int64_t carry0;
24
25    int64_t carry1;
26
27    int64_t carry2;
28
29    int64_t carry3;
30
31    int64_t carry4;
32
33    int64_t carry5;
34
35    int64_t carry6;
36
37    int64_t carry7;
38
```



```
39     int64_t carry8;
40
41     int64_t carry9;
42
43     /* Validate the number to be canonical */
44     if (h9 == 33554428 && h8 == 268435440 && h7 == 536870880 && h6 ==
2147483520 &&
45         h5 == 4294967295 && h4 == 67108860 && h3 == 134217720 && h2 ==
536870880 &&
46         h1 == 1073741760 && h0 >= 4294967277) {
47         return -1;
48     }
49
50     carry9 = (h9 + (int64_t)(1 << 24)) >> 25; h0 += carry9 * 19; h9 -=
carry9 << 25;
51     carry1 = (h1 + (int64_t)(1 << 24)) >> 25; h2 += carry1; h1 -= carry1
<< 25;
52     carry3 = (h3 + (int64_t)(1 << 24)) >> 25; h4 += carry3; h3 -= carry3
<< 25;
53     carry5 = (h5 + (int64_t)(1 << 24)) >> 25; h6 += carry5; h5 -= carry5
<< 25;
54     carry7 = (h7 + (int64_t)(1 << 24)) >> 25; h8 += carry7; h7 -= carry7
<< 25;
55
56     carry0 = (h0 + (int64_t)(1 << 25)) >> 26; h1 += carry0; h0 -= carry0
<< 26;
57     carry2 = (h2 + (int64_t)(1 << 25)) >> 26; h3 += carry2; h2 -= carry2
<< 26;
58     carry4 = (h4 + (int64_t)(1 << 25)) >> 26; h5 += carry4; h4 -= carry4
<< 26;
59     carry6 = (h6 + (int64_t)(1 << 25)) >> 26; h7 += carry6; h6 -= carry6
<< 26;
60     carry8 = (h8 + (int64_t)(1 << 25)) >> 26; h9 += carry8; h8 -= carry8
<< 26;
61
62     y[0] = h0;
63     y[1] = h1;
64     y[2] = h2;
65     y[3] = h3;
66     y[4] = h4;
67     y[5] = h5;
68     y[6] = h6;
69     y[7] = h7;
```



```

70     y[8] = h8;
71     y[9] = h9;
72
73     return 0;
74 }

```

Table 5: fe_frombytes_vartime Source To Spec Map

L1	L2	Code	Purpose
1384	1	<code>h0 = load_4(s)</code>	Unpack bits [0, 31] into limb 0
1385	2	<code>h1 = load_3(s+4) << 6</code>	Unpack bits [32, 55], shift for limb 1 position
1386-1392	2	<code>h2...h8 loads</code>	Unpack and shift remaining limbs
1393	3	<code>h9 = (load_3(s+29) & 8388607) << 2</code>	Top limb: mask 0x7FFFFFFF clears bit 255. (1) <i>The ED25519 sign bit.</i>
1405-1409	2	<code>Canonicality if</code>	Check if $v \geq p$. (2)
1412	4	<code>carry9 = ... h0 += carry9 * 19</code>	Top-limb carry wraps: $2^{255} \equiv 19 \pmod{p}$
1412-1422	2	<code>Carry chain</code>	Odd limbs carry at 2^{25} , even at 2^{26} . (3) <i>Two pass: odds, then evens.</i>
1424-1433	2	<code>y[i] = h_i</code>	Store reduced limbs
1435	7	<code>return 0</code>	Success

Located in crypto-ops.c

4.4.2 Implementation Observations

1. The sign bit inherited by the ED25519 encoding is cleared. For a standalone field element deserialization, this means that bytes with bit 255 set are silently treated as if they were clear.
2. The **Canonicality if** step checks by comparing pre-carry limbs against p 's limb representation: limbs $h_1, \dots, h_9$ are tested for strict equality against the encoded limbs of p , while h_0 uses $h_0 \geq 2^{32} - 19$ since the bottom 32 bits of p equal $2^{32} - 19$. If all comparisons pass, the function returns -1 .



3. This functions identically to `fe_frombytes`, with the exception of the canonicity check. `fe_frombytes` will reduce inputs modulo p , but `fe_frombytes_vartime` will reject them.

4.4.3 Edge Case Analysis

Input	Behavior
All zero bytes	Returns 0
Bytes encoding p	Returns -1
Bytes encoding $p + 1, \dots, 2^{225} - 2$	Returns -1
Bytes encoding $2^{225} - 1$	Returns -1
Bytes with bit 255 set, low 255 bits $< p$	Returns 0; bit 255 silently masked at line 1393.
Bytes with bit 255 set, low 255 bits $\geq p$	Returns -1 : bit 255 is masked, then the canonicity check fires on the remaining limbs.
$p - 1$	Returns 0

4.4.4 Fuzzing Results

`fe_frombytes_vartime` is exercised indirectly by every `afl_x25519_conv` iteration, since `edwards_bytes_to_x25519_vartime` deserialises the byte input through it. Under MSan the harness ran 202,703,472 iterations with zero uninitialised-memory reads; under ASan/UBSan/CMPLOG/extended-ASan no memory-safety findings emerged. The canonicity check (return -1 on $b \geq p$) was confirmed to fire deterministically on near-prime inputs: the rejection logic at lines 1405–1409 fired on every `0xff...ff7f`-style input the fuzzer generated.



4.5 Section Conclusion



All functions used for the *Batch Inverse Section* are described correctly mathematically, and implemented according to that description. As such, it is our conclusion that the above functions are fit for deployment in Monero.



5 ED25519 to X25519 Point Conversion

The purpose of this section will be to determine the following:

1. From a cryptographic standpoint, do the functions properly derive the X25519 x -coordinate from an ED25519 point?
2. Is the code in each function implemented correctly assuming the above is true?

5.1 Mathematical Overview

Ed25519, in its Twisted Edwards form, is defined as

$$-x^2 + y^2 = 1 + dx^2y^2 \text{ over } \mathbb{F}_p$$

where $p = 2^{255} - 19$ and $d = -\frac{121665}{121666}$.

Curve25519, in its Montgomery form, is given as the polynomial:

$$v^2 = u^3 + 486662u^2 + u \text{ over } \mathbb{F}_p$$

It is known that these two curves are *birationally equivalent*, which loosely means that, with the exception of a small subset of points, the curves are isomorphic to each other. So given a point (x, y) on the first curve, one can determine the corresponding coordinate on the second curve using the map

$$u = \frac{1 + y}{1 - y},$$

which is valid for all values except $y = 1$ (the Edwards identity, where the denominator vanishes).

The following functions are used to compute this conversion.



5.2 PR #9828 - edwardsYZ_to_x25519

5.2.1 Functionality

Listing 6: edwardsYZ_to_x25519 Code

```
1 static void edwardsYZ_to_x25519(unsigned char* xbytes, const fe Y, const
   fe Z) {
2     fe tmp0;
3     fe tmp1;
4     fe_add(tmp0, Z, Y);
5     fe_sub(tmp1, Z, Y);
6     fe_invert(tmp1, tmp1);
7     fe_mul(tmp0, tmp0, tmp1);
8     fe_tobytes(xbytes, tmp0);
9 }
```

Table 6: edwardsYZ_to_x25519 Source To Spec Map

L1	L2	Code	Purpose
3846	1	fe_add(tmp0, Z, Y)	tmp0 = Z + Y
3847	2	fe_sub(tmp1, Z, Y)	tmp1 = Z - Y
3848	3	fe_invert(tmp1, tmp1)	Set tmp1 as $(tmp1^{p-2} \bmod p)$
3849	4	fe_mul(tmp0, tmp0, tmp1)	Set tmp0 = tmp0 · tmp1
3850	5	fe_tobytes(xbytes, tmp0)	Serialize tmp0 to 32-byte LE

Located in crypto-ops.c

5.2.2 Implementation Observations

1. When $Z = Y$, the inversion via Fermat's Little Theorem returns 0 (the `fe_invert(0)=0` artifact), so the function silently outputs $u = 0$ on Montgomery: the unique 2-torsion point on Curve25519, not the identity. No spec documents this fallthrough, and downstream callers who consume the output without further validation would be exposed to a small-subgroup gadget.



5.2.3 Edge Case Analysis

Outside of the implementation note, there is no edge case analysis necessary. It is a simple application of fundamental elliptic curve operations and so long as the inversion is respected (non-zero), it should run without issue.

5.2.4 Fuzzing Results

`edwardsYZ_to_x25519` is the inner kernel of every `afl_x25519_conv` iteration: both the struct path (`ge_p3_to_x25519`) and the byte path (`edwards_bytes_to_x25519_vartime`) terminate in this function. Across the libFuzzer (1,559,634 iters), AFL++ ASan (1,320), CMPLOG (1,627), MSan (202,703,472), and extended-ASan (10,099) rounds the function ran for over 204.3M sanitiser-instrumented executions with zero memory-safety findings. The $Z = Y$ degenerate case (which silently returns the identity) was not surfaced by any campaign: inputs come from real curve points where $Z = Y$ requires y -coordinate equal to the projective z , which is not produced by `ge_frombytes_vartime` on canonical inputs.



5.3 PR #9828 - ge_p3_to_x25519

5.3.1 Functionality

Listing 7: ge_p3_to_x25519 Code

```
1 void ge_p3_to_x25519(unsigned char* xbytes, const ge_p3* h)
2 {
3     edwardsYZ_to_x25519(xbytes, h->Y, h->Z);
4 }
```

Table 7: ge_p3_to_x25519 Source To Spec Map

L1	L2	Code	Purpose
3855	3	edwardsYZ_to_x25519(xbytes, h->Y, h->Z)	Extracts the Y and Z coordinate from (X:Y:Z:T).

Located in crypto-ops.c

5.3.2 Implementation Observations

1. There is no validation mechanism and the function ‘assumes’ it was validated at construction.

5.3.3 Fuzzing Results

ge_p3_to_x25519 is the *struct-path* side of the differential libFuzzer harness fuzz_diff_x25519_hacl, which compares its output byte-for-byte against HACL*’s formally-verified `Hacl_Curve25519_51_scalarmult` on a G -derived point. Across 408,926 random Ed25519 secret keys (80,682 coverage regions spanning both Monero crypto-ops and HACL*) zero divergences were observed, and the campaign produced no ASan or UBSan findings beyond the pre-existing signed-left-shift behavior in `fe_mul/fe_sq` that is a known Monero artifact. The function thus produces the same u -coordinate as a formally-verified reference on every tested input.



5.4 PR #9828 - edwards_bytes_to_x25519_vartime

5.4.1 Functionality

Listing 8: edwards_bytes_to_x25519_vartime Code

```
1 int edwards_bytes_to_x25519_vartime(unsigned char* xbytes, const unsigned
   char* s)
2 {
3     fe Y;
4     if (fe_frombytes_vartime(Y, s) != 0) {
5         return -1;
6     }
7
8     fe Z;
9     fe_1(Z);
10
11     edwardsYZ_to_x25519(xbytes, Y, Z);
12
13     return 0;
14 }
```

Table 8: edwards_bytes_to_x25519_vartime Source To Spec Map

L1	L2	Code	Purpose
3861	4	fe_frombytes_vartime(Y, s)	Deserialize 32 byte s to field element Y
			Reject if $Y \geq p$
3862	5	return -1	Non-canonical input rejection
3866	9	fe_1(Z)	Z = 1 Input is affine, so $y = Y$)
3868	11	edwardsYZ_to_x25519(xbytes, Y, Z)	Set xbytes = $\frac{1+Y}{1-Y}$

Located in crypto-ops.c

5.4.2 Implementation Observations

1. The projective formula avoids a field inversion in comparison to the naive approach. This optimization is not in the birational map spec.



2. The variable time behavior of this function comes from `fe_frombytes_vartime`, however, the rest of the function is constant time.
3. The function will check the canonical encoding but does not verify the y coordinate falls on the curve. Contrast `ge_frombytes_vartime`, which does perform x -recovery and the curve-equation check $-x^2 + y^2 = 1 + dx^2y^2$; the byte path here accepts any canonical $y \in [0, p)$ regardless of whether a real Ed25519 point exists with that y .
4. Combined with the silent-identity behavior of `edwardsYZ_to_x25519`, an adversary who supplies the canonical Edwards-identity encoding `01 00...00` obtains $u = 0$ on Montgomery, which is the 2-torsion point. Any future caller that consumes the output without further validation would be exposed to a small-subgroup gadget. The current PR introduces no callers, so the issue is an API precondition rather than a present-day exposure; a hardened variant should reject when `fe_iszero(Z - Y)`.

5.4.3 Edge Case Analysis

Input (32-byte y encoding)	Behavior
y encodes $(0, 1)$ (Edwards identity)	Output $u = 0$ (the 2-torsion point); int return 0.
y encodes $(0, -1)$ (4-torsion)	Output $u = 0$; int return 0.
y encodes a point with y -coordinate 0	Output $u = 1$; int return 0.
y encodes the basepoint G	Output $u = 9$ (the Curve25519 basepoint); int return 0.
$y \geq p$ (non-canonical)	Int return -1 ; output u undefined.
Canonical $y < p$ on a curve point	Output $u = (1 + y)/(1 - y)$ on Montgomery; int return 0.

5.4.4 Fuzzing Results

The CMPLOG round of `af1_x25519_conv` surfaced 109 SIGILL property violations in which the byte path (`edwards_bytes_to_x25519_vartime`) and the struct path (`ge_p3_to_x25519`) disagreed on the resulting u -coordinate. Triage classified all 109 as input-precondition divergences: `ge_frombytes_vartime` silently reduces non-canonical $y \geq p$ during decompression, while the byte path does not reduce $y \bmod p$ before computing $\frac{1+y}{1-y}$, so the two paths land at different limb representations on the same near-prime input. For canonical inputs (the only inputs the protocol can produce) the two paths agree byte-for-byte across the 204.3M MSan iterations summarised above, and the campaign reported zero ASan/UBSan/MSan



memory-safety findings. A defense-in-depth hardening opportunity which follows would be to have the byte path reduce $y \bmod p$ before the projective formula, restoring parity with the struct path.

5.5 Section Conclusion



Some care must be taken regarding the `edwardsYZ_to_x25519` kernel covered earlier in this section, since it will work exactly as described so long as the implicit hypothesis $Z \neq Y$ is respected at the call site. Specifically, when $Z = Y$ the inversion via Fermat's Little Theorem returns 0 and the function silently emits $u = 0$ (the unique non-trivial 2-torsion point on Curve25519); a hardened variant should reject when `fe_iszero(Z - Y)`. Beside this detail, the mathematics which the code describes are sound and the code implements it correctly.



6 Torsion Clearing

The goals of this section are to answer the following questions:

1. Does each function, when called, soundly perform as desired in a cryptographic sense?
2. Is the code in each function implemented correctly?
3. Are `clear_torsion` and `mul8_is_identity` constant time?

In this section, we will analyze the functions `clear_torsion`, `mul8_is_identity`, `verPointsForTorsion` and `get_valid_torsion_cleared_point`. The focus of the first three subsections will be looking at the code as it is written and ensuring that each of the functions does what it is intended to do.

6.1 Mathematical Overview

When dealing with a non-prime order group, it is not guaranteed that a randomly chosen point will fall in a prime order subgroup. In this particular instance, the ED25519 curve has order $|\mathbb{E}(\mathbb{F}_p)| = 8\ell$ where $\ell = 2^{252} + 27742317777372353535851937790883648493$, which is prime. The purpose of torsion clearing is to make sure that values do not inadvertently fall into the subgroup of order 8, as many of the security properties that ED25519 enjoys are not valid in the smaller subgroups.

In order to be explicit, the torsion subgroup of ED25519, which we will call $\mathbb{E}_8$, we represent as $\mathbb{E}_8 = \{\mathcal{O}, T_1, \dots, T_7\}$. Each of the points are defined as follows:

Point	Coordinates	Order
$\mathcal{O}$	$(0, 1)$	1
T_1	$(0, -1)$	2
T_2	$(1, 0)$	4
T_3	$(-1, 0)$	4
T_4	(ρ, γ)	8
T_5	$(-\rho, \gamma)$	8
T_6	$(\rho, -\gamma)$	8
T_7	$(-\rho, -\gamma)$	8



where

$$\rho = 2^{\frac{p-1}{4}} \bmod p \quad \text{and} \quad \gamma = \left(\frac{1+\rho}{2}\right)^{\frac{1}{2}} \bmod p.$$

In order to ensure points are not in $\mathbb{E}_8$, the following functions are used:

1. `mul8_is_identity(P)`

- Computes $8 \cdot P$.
- Checks if $8 \cdot P = 0$

2. `clear_torsion(P)`

- Given $P = P_0 + T$, computes P_0 .
- The above form is the decomposition of P into an element of the prime order subgroup and the torsion subgroup.

3. `get_valid_cleared_torsion_point(P,out)`

The first function `mul8_is_identity` acts exactly how one would expect: it computes $8 \cdot P$ for input point P , then determines whether the point is the identity or not. It does so via 3 point doublings, by computing $2 \cdot P \rightarrow 2 \cdot (2 \cdot P) = 4 \cdot P \rightarrow 2 \cdot (4 \cdot P) = 8 \cdot P$. It checks whether or not the output is the identity element as follows:

1. The computed coordinates are converted to extended coordinates $(X : Y : Z : T)$.
2. Determine if the point is equal to $(0 : c : c : 0)$, for some non-zero scalar c .
3. If so, then the point is the identity element, otherwise it is not.

The second function `clear_torsion` is slightly more mathematically involved. Since the curve is a finitely generated group G , we can directly apply the *fundamental theorem of finite abelian groups* to see that G is isomorphic to the direct sum of cyclic subgroups of prime power order.

Since $\mathbb{E}(\mathbb{F}_p) = 8\ell$, this curve will be isomorphic to:

$$\mathbb{Z}/8\mathbb{Z} \oplus \mathbb{Z}/\ell\mathbb{Z}$$

This implies that every point P on the curve can be expressed as the sum of two points P_0 and T , where:

$$P = P_0 + T$$



where P_0 is an element of order ℓ and T is an element of order dividing 8.

In order to retrieve the P_0 point, we can perform the following procedure:

1. Compute $8 \cdot P$. Simplifying the computation, we have:

$$8 \cdot P = 8 \cdot (P_0 + T) = 8 \cdot P_0 + 8 \cdot T$$

2. Since T has an order dividing 8, we have that:

$$8 \cdot T = 0 \Rightarrow 8 \cdot P = 8 \cdot P_0$$

3. Since P_0 has order ℓ , it's a fact that $8 \cdot P_0$ remains in the prime subgroup. As such, 8^{-1} exists modulo ℓ . So, we multiply by the inverse, resulting in

$$8^{-1} \cdot (8 \cdot P) = P_0,$$

where all operations take place in the appropriate modulus. Subsequently, the element produced is guaranteed to be an element of the prime order subgroup exclusively.

The last function simply puts these two pieces together.



6.2 PR #10342 - mul8_is_identity

6.2.1 Functionality

Listing 9: mul8_is_identity Code

```
1 bool mul8_is_identity(const ge_p3& point) {
2     ge_p2 point_ge_p2;
3     ge_p3_to_p2(&point_ge_p2, &point);
4     ge_p1p1 point_mul8;
5     ge_mul8(&point_mul8, &point_ge_p2);
6     ge_p3 point_mul8_p3;
7     ge_p1p1_to_p3(&point_mul8_p3, &point_mul8);
8     return ge_p3_is_point_at_infinity_vartime(&point_mul8_p3);
9 }
```

Table 9: mul8_is_identity Source To Spec Map

L1	L2	Code	Purpose
34	1	bool mul8_is_identity(const ge_p3 &point)	Predicate: $8P = O$
36	3	ge_p3_to_p2(&point_ge_p2, &point)	$(X : Y : Z : T) \Rightarrow (X : Y : Z)$
38	5	ge_mul8(&point_mul8, &point_ge_p2)	$8P$ via three point doublings.
40	7	ge_p1p1_to_p3(&point_mul8_p3, &point_mul8)	Convert to extended form.
41	8	return ge_p3_is_point_at_infinity_vartime(...)	Checks if $X = 0, T = 0, Y = Z$ and
			$Y \neq 0$. True iff $(0 : c : c : 0) = O$.

Located in fcmp_pp_crypto.cpp

6.2.2 Implementation Observations

1. The function does not carry a `_vartime` suffix. The variable runtime is not problematic since all inputs are public data, but should be included for the sake of clarity and consistency.



6.2.3 Edge Case Analysis

Input	Behavior
$\mathbb{O}$	true
$T_1, \dots, T_7$	true
$G + T$	false
$y \geq p$	Not reachable: <code>mul8_is_identity</code> takes a <code>ge_p3</code> , so non-canonical $y \geq p$ would be rejected upstream by the caller's <code>ge_frombytes_vartime</code> before invocation.
Off-curve bytes	Same as above: rejection is upstream of this function.

6.2.4 Fuzzing Results

`mul8_is_identity` is the small-order rejection step of `afl_torsion` and is invoked on every queue input across the ASan (4,078), CMPLOG (10,147), and extended-ASan (10,344) rounds. ASan reported no memory-safety findings. The campaign confirmed correct rejection on the eight torsion points and acceptance on $G + T$ inputs; the property-violation crashes recorded against `afl_torsion` are attributable to `clear_torsion` idempotency rather than to `mul8_is_identity` itself.



6.3 PR #10342 - clear_torsion

6.3.1 Functionality

Listing 10: clear_torsion Code

```
1 crypto::ec_point clear_torsion(const ge_p3& point) {
2     ge_p2 point_inv_8;
3     ge_scalarmult(&point_inv_8, to_bytes(crypto::EC_INV_EIGHT), &point);
4     ge_p1p1 point_inv_8_mul_8;
5     ge_mul8(&point_inv_8_mul_8, &point_inv_8);
6     ge_p3 torsion_cleared_point;
7     ge_p1p1_to_p3(&torsion_cleared_point, &point_inv_8_mul_8);
8     crypto::ec_point k_out;
9     ge_p3_tobytes(to_bytes(k_out), &torsion_cleared_point);
10    return k_out;
11 }
```

Table 10: clear_torsion Source To Spec Map

L1	L2	Code	Purpose
44	1	clear_torsion(const ge_p3 &point)	$P \Rightarrow P_0$ where $P = P_0 + T$
46-47	3-4	ge_scalarmult(&point_inv_8, to_bytes,...)	$Q = (8^{-1} \bmod \ell) \cdot P$
48-49	5	ge_mul8(&point_inv_8_mul_8, &point_inv_8)	$R = 8 \cdot (8^{-1} \cdot P) = P_0 + 8T' = P_0$
50-51	7	ge_p1p1_to_p3(&torsion_cleared_point, ...)	Convert to extended for serialization
52-53	9	ge_p3_tobytes(to_bytes(k_out), ...)	Compress to 32-byte canonical encoding

Located in fcmap_pp_crypto.cpp

6.3.2 Implementation Observations

1. This function also does not carry a `_var_time` suffix, despite calling `ge_scalarmult` which is variable time. However, like the prior function, this is not problematic from a security standpoint, since the inputs are public.
2. The constant `EC_INV_EIGHT` used at Line 3 is declared with type `crypto::ec_point` in `crypto.h`, but is semantically a scalar (the value $8^{-1} \bmod \ell$, byte-identical to `rct::INV_EIGHT`).



Use as a scalar is laundered through the `to_bytes(ec_point)` overload, which performs a `reinterpret_cast` to `const unsigned char*`. Pre-FCMP++ Monero stores the same constant as an untyped `rct::key` (deliberately type-agnostic); the new `ec_point` alias is a regression in auditability rather than a behavioural bug. Re-typing as `ec_scalar` would have zero functional impact and remove the cast.

6.3.3 Edge Case Analysis

Input	Behavior
$\mathbb{O}$	0
$T_1, \dots, T_7$	0
$G + T$	G
$y \geq p$	Not reachable: <code>clear_torsion</code> takes a <code>ge_p3</code> , so non-canonical $y \geq p$ would be rejected upstream by the caller's <code>ge_frombytes_vartime</code> before invocation.
Off-curve bytes	Same as above: rejection is upstream of this function.

6.3.4 Fuzzing Results

The AFL++ harness `afl_torsion` exercised `clear_torsion` for 20,969 total iterations across the ASan, CMPLOG, and extended-ASan rounds. AFL++ recorded 50 SIGILL property violations (27 ASan + 23 CMPLOG, plus 14 in the extended-ASan round) when its `__builtin_trap()` assertion fired on either the idempotency check:

$$(\text{clear_torsion}(\text{clear_torsion}(P)) = \text{clear_torsion}(P))$$

or the small-order-identity check. All crashes triaged to a single root cause: non-canonical encodings with $y \geq p = 2^{255} - 19$. `ge_frombytes_vartime` is permissive about $y \geq p$ (it reduces silently during decompression), so the resulting limb values then diverge from those produced by re-running the pipeline on the cleared output. Protocol callers always supply canonical encodings, so the violations represent input-precondition gaps rather than function bugs. Zero ASan memory-safety findings.



6.4 PR #10342 - get_valid_cleared_torsion_point

6.4.1 Functionality

Listing 11: get_valid_cleared_torsion_point Code

```
1 bool get_valid_torsion_cleared_point(const crypto::ec_point& point, crypto
  ::ec_point& torsion_cleared_out) {
2     ge_p3 p3;
3     if (ge_frombytes_vartime(&p3, to_bytes(point)) != 0)
4         return false;
5     if (mul8_is_identity(p3))
6         return false;
7     torsion_cleared_out = fcmp_pp::clear_torsion(p3);
8     if (torsion_cleared_out == crypto::EC_I)
9         return false;
10    return true;
11 }
```

Table 11: get_valid_cleared_torsion_point Source To Spec Map

L1	L2	Code	Purpose
57	1	bool get_valid_torsion_cleared_point(...)	Definition Statement
59	3	ge_frombytes_vartime(...) != 0	Decompress 32 bytes to curve point.
			Rejects off-curve and non-canonical $y \geq p$
61	5	mul8_is_identity(p3)	Reject if $8P = O$ (all 8 torsion elements)
63	7	fcmp_pp::clear_torsion(p3)	Project onto prime-order subgroup.
64	8	torsion_cleared_out == crypto::EC_I	Reject if torsion-cleared result is identity.

Located in fcmp_pp_crypto.cpp

6.4.2 Implementation Observations

1. It is not entirely clear why the function has a redundant check for the torsion cleared output (Line 8 in the included code). If a torsion cleared element were to be the identity, then it would imply that the original element would have been rejected by `mul8_is_identity`.



6.4.3 Edge Case Analysis

Since this function is simply iterative calling of the prior functions, there is no edge case analysis necessary, since all problematic instances would be covered by the previous tables given.

6.4.4 Fuzzing Results

`get_valid_torsion_cleared_point` is the validation pipeline that wraps `ge_frombytes_vartime`, `mul8_is_identity`, and `clear_torsion`; it is fuzzed transitively through `afl_output_to_tuple`, which routes every queue input through it on the legacy-output branch. Across the ASan (308,443) and CMPLOG (402,193) rounds (710,636 iterations combined) zero ASan memory-safety findings were recorded, and the explicit identity-rejection branch was reached without further crashes on any reduced torsion input.



6.5 Section Conclusion



To summarize:

- Each of the above listed functions performs mathematically soundly.
- Each of the above functions appear to be implemented correctly in the code.
- `clear_torsion` and `mul8_is_identity` are certainly **not** constant time. `clear_torsion` has an instance of `ge_scalarmult` in its body, which is not a constant time function. `mul8_is_identity` is also not constant time since it calls a `_vartime` suffixed function in its body. However, both of these are acceptable, since the inputs are public data, so nothing is gained from timing attacks.



7 Change to `fe_reduce_vartime`

The purpose of this section is to address the following:

1. Is the rationale documented in the source comment immediately preceding `fe_reduce_vartime` (`crypto-ops.c`, lines 3887–3906) valid rationale for introducing the function? The comment states that `fe_add` and `fe_sub` have postcondition bounds ($|h|$ bounded by $1.1 \cdot 2^{26}, 1.1 \cdot 2^{25}, \dots$) that are looser than their precondition bounds ($|f|, |g|$ bounded by $1.1 \cdot 2^{25}, 1.1 \cdot 2^{24}, \dots$), so a chain of additions/subtractions can leave a field element outside the input domain of the next call; `fe_reduce_vartime` contracts the limbs back into the precondition domain by round-tripping through `fe_tobytes` and `fe_frombytes_vartime`.
2. Is `fe_reduce_vartime` sound from a cryptographic standpoint?
3. Is the code introduced by the PR implemented correctly assuming the cryptography is sound?

7.1 PR #10135 - `fe_reduce_vartime`

7.1.1 Functionality

Listing 12: `fe_reduce_vartime` Code

```
1 int fe_reduce_vartime(fe reduced_f, const fe f)
2 {
3     unsigned char f_bytes[32];
4     fe_tobytes(f_bytes, f);
5     return fe_frombytes_vartime(reduced_f, f_bytes);
6 }
```

7.1.2 Implementation Observations

1. There is a minor cosmetic typo on line 3900, where ‘poscondition’ should be ‘postcondition’.

Table 12: fe_reduce_vartime Source To Spec Map

L1	L2	Code	Purpose
3907	1	<code>int fe_reduce_vartime(...)</code>	Returns 0 on success, -1 on non-canonical input
3909	3	<code>unsigned char f_bytes[32]</code>	Allocate b in $\{0, 1\}^{256}$
3910	4	<code>fe_tobytes(f_bytes, f)</code>	$b = \text{canonical_encode}(f \bmod p)$
3911	5	<code>return fe_frombytes_vartime(...)</code>	$f' = \text{decode}(b)$ with limbs in precondition bounds
			Reject if $b \geq p$

Located in crypto-ops.c

7.1.3 Edge Case Analysis

Input	Behavior	Output
Limbs encoding 0	Bytes = 0x00...00, reloads as zero limbs	0
Limbs encoding $p = 2^{255} - 19$	<code>fe_tobytes</code> reduces to 0, Bytes = 0x00...00	0
Limbs encoding $p + 1$	<code>fe_tobytes</code> reduces to 1	0
Limbs at max postcondition bound	<code>fe_tobytes</code> carry chain handles this	0
(1.12^{26} even, 1.12^{25} odd)		
Limbs exceeding postcondition bound	<code>fe_tobytes</code> carry chain may overflow <code>int32_t</code>	Undefined
(Near 2^{31})		
Bytes $\geq p$ after <code>fe_tobytes</code>	<code>fe_tobytes</code> always outputs $[0, p)$	Unreachable

7.1.4 Fuzzing Results

The function's two callees, `fe_tobytes` and `fe_frombytes_vartime`, are the same routines exercised by `afl_x25519_conv` for 202,703,472 MSan iterations with zero memory-safety findings, so the round-trip semantics of `fe_reduce_vartime` inherit that coverage. A regression test `FeReduceVartime.RoundTrip` enumerates the postcondition limb bounds described in the source-to-spec map above; running individual PR-branch builds remains a follow-up to close the dedicated-harness gap.



7.2 Section Conclusion



As it is written above, it is our opinion the following code is fit for use. The round-trip through `fe_tobytes` and `fe_frombytes_vartime` correctly contracts limb magnitudes back into the precondition domain of `fe_add/fe_sub`.



8 ED25519 to Weierstrass Point Conversion

The purpose of this section is as follows:

1. Do the functions correctly implement the specification detailed in section E.2 of the paper linked:
<https://www.ietf.org/archive/id/draft-ietf-lwig-curve-representations-02.pdf>?
2. Do the constants introduced (`fe_a`, `fe_a_inv_3`, `fe_c`) match the constants specified in the paper?
3. Is `fe_ed_derivatives_to_wei_x_y` constant time?

8.1 Mathematical Overview

The following functions are used to convert elliptic curve coordinates from ED25519 in the Twisted Edwards form to short Weierstrass form, which is used in the FCMP++ tree construction. Since both the Selene and Helios curves are in short Weierstrass form, every output public key must be converted before it can be used as a leaf in the curve tree.

In order to facilitate this, we convert first from Twisted Edwards coordinates to Montgomery coordinates, then from Montgomery to short Weierstrass form. The first conversion is done in the usual way (as detailed in PR #9828), by setting:

$$u = \frac{1+y}{1-y} \quad \text{and} \quad v = \frac{1+y}{(1-y) \cdot x}$$

The second conversion is also done in the usual way:

$$w_x = u + \frac{A}{3} \quad \text{and} \quad w_y = c \cdot v$$

where the constants A , $\frac{A}{3} \pmod{p}$, and c are hardcoded constants, defined in `crypto-ops-data.c` as:

- The A coefficient: `fe_a` = 486662.
- The $\frac{A}{3} \pmod{p}$ component: `fe_a_inv_3` (Stored as a 10-limb radix field element).
- The c coefficient: `fe_c` = $\sqrt{-(A+2)} \pmod{p}$,

where $p = 2^{255} - 19$.



8.2 PR #10345 - fe_ed_derivatives_to_wei_x

8.2.1 Functionality

Listing 13: fe_ed_derivatives_to_wei_x Code

```
1 static void fe_ed_derivatives_to_wei_x(unsigned char* wei_x, const fe
    inv_one_minus_y, const fe one_plus_y)
2 {
3     // (1/(1-y))*(1+y)
4     fe inv_one_minus_y_mul_one_plus_y;
5     fe_mul(inv_one_minus_y_mul_one_plus_y, inv_one_minus_y, one_plus_y);
6
7     // wei_x = (1/(1-y))*(1+y) + (A/3)
8     fe wei_x_fe;
9     fe_add(wei_x_fe, inv_one_minus_y_mul_one_plus_y, fe_a_inv_3);
10    fe_tobytes(wei_x, wei_x_fe);
11 }
```

Table 13: fe_ed_derivatives_to_wei_x Source To Spec Map

L1	L2	Code	Purpose
3937	1	Parameters : inv_one_minus_y, one_plus_y	Inputs: $(1 - y)^{-1}, (1 + y)$
3941	3	fe_mul(..., inv_one_minus_y, one_plus_y)	$u = \frac{(1+y)}{(1-y)}$
			Montgomery u-coordinate
3945	4	fe_add(wei_x_fe, ..., fe_a_inv_3)	$w_x = u + \frac{A}{3}$
			Montgomery to Weierstrass
3946	5	fe_tobytes(wei_x, wei_x_fe)	Serialize, implicitly reduces mod p .

Located in crypto-ops.c

8.2.2 Implementation Observations

1. This function is static and only callable within `crypto-ops.c`, where it is called by `fe_ed_derivatives_to_wei_x_y`.



8.3 PR #10345 - fe_ed_derivatives_to_wei_x_y

8.3.1 Functionality

Listing 14: fe_ed_derivatives_to_wei_x_y Code

```
1 void fe_ed_derivatives_to_wei_x_y(unsigned char* wei_x, unsigned char*
   wei_y, const fe inv_one_minus_y, const fe one_plus_y, const fe
   inv_one_minus_y_mul_x)
2 {
3   fe_ed_derivatives_to_wei_x(wei_x, inv_one_minus_y, one_plus_y);
4
5   // c*(1+y)
6   fe fe_c_mul_one_plus_y;
7   fe_mul(fe_c_mul_one_plus_y, fe_c, one_plus_y);
8
9   // wei_y = c * (1+y) * (1/((1-y)*x))
10  fe wei_y_fe;
11  fe_mul(wei_y_fe, fe_c_mul_one_plus_y, inv_one_minus_y_mul_x);
12  fe_tobytes(wei_y, wei_y_fe);
13 }
```

Table 14: fe_ed_derivatives_to_wei_x_y Source To Spec Map

L1	L2	Code	Purpose
3950	1	Parameters : inv_one_minus_y	Inputs: $(1 - y)^{-1}, (1 + y), ((1 - y) \cdot x)^{-1}$
		one_plus_y, inv_one_minus_y_mul_x	
3952	3	fe_ed_derivatives_to_wei_x(...)	Delegates w_x computation to the prior function
3956	7	fe_mul(..., fe_c, one_plus_y)	$c \cdot (1 + y)$
3960	11	fe_mul(wei_y_fe, ...)	$w_y = \frac{c \cdot (1+y)}{(1-y) \cdot x}$
3961	12	fe_tobytes(wei_y, wei_y_fe)	Serialize, implicitly reduces mod p

Located in crypto-ops.c

8.3.2 Implementation Observations

There are no important notes or observations regarding this function.



8.3.3 Edge Case Analysis

Input	Behavior
Identity ($y = 1$)	Caller rejects via <code>point_to_ed_derivatives</code> ; never reached.
$x = 0$	$(1 - y) \cdot x = 0$ inverse undefined; rejected upstream.
Generator G	Returns canonical (w_x, w_y) on the Weierstrass curve.
Torsion / near-prime y	Canonical limbs satisfy $w_y^2 = w_x^3 + a_{\text{wei}}w_x + b_{\text{wei}} \pmod{p}$.
Non-canonical $y \geq p$	Output non-reduced; harness curve-equation check fails (see fuzzing).

8.3.4 Fuzzing Results

The AFL++ harness `afl_wei_conv` exercised the full Weierstrass conversion pipeline (`point_to_ed_derivatives` $\rightarrow$ `batch invert` $\rightarrow$ `fe_ed_derivatives_to_wei_x_y`) for 29,078 iterations across the ASan (15,773), CMPLOG (1,276), and extended-ASan (12,029) rounds. AFL++ recorded 66 SIGILL property violations (25 ASan + 23 CMPLOG + 18 extended-ASan) where the harness' Weierstrass curve-equation check $w_y^2 = w_x^3 + a_{\text{wei}}w_x + b_{\text{wei}} \pmod{p}$ failed. All 66 triaged to a single root cause: non-canonical encodings with $y \geq p$ produce non-reduced limb values that propagate through `fe_mul` and `fe_add`, yielding w_x, w_y that satisfy a shifted equation. The conversion is correct on every canonical input. Zero ASan memory-safety findings; harness Weierstrass constants $a_{\text{wei}}, b_{\text{wei}}$ were independently verified and are not the cause of the property violations.



8.4 PR #10345 - point_to_ed_derivatives

8.4.1 Functionality

Listing 15: point_to_ed_derivatives Code

```
1 bool point_to_ed_derivatives(const crypto::ec_point& pub, EdDerivatives&
   ed_derivatives) {
2     if (pub == crypto::EC_I)
3         return false;
4     // fe y;
5     ge_p3 p3;
6     if (ge_frombytes_vartime(&p3, to_bytes(pub)) != 0)
7         return false;
8     fe one;
9     fe_1(one);
10    // (1+y),(1-y)
11    fe_add(ed_derivatives.one_plus_y, one, p3.Y);
12    fe_sub(ed_derivatives.one_minus_y, one, p3.Y);
13    // (1-y) * x
14    fe_mul(ed_derivatives.one_minus_y_mul_x, ed_derivatives.one_minus_y,
   p3.X);
15    return true;
16 }
```



Table 15: point_to_ed_derivatives Source To Spec Map

L1	L2	Code	Purpose
70	1	<code>if (pub == crypto::EC_I) return false</code>	Reject $(0, 1)$. <i>Would make $1 - y = 0$.</i>
74	3	<code>ge_frombytes_vartime(&p3, to_bytes(pub))</code>	Decompress point. <i>Rejects non-canonical $y \geq p$.</i>
77	7	<code>fe_1(one)</code>	Calls 1.
79	11	<code>fe_add(ed_derivatives.one_plus_y, one, p3.Y)</code>	Computes $(1 + y)$.
80	12	<code>fe_sub(ed_derivatives.one_minus_y, one, p3.Y)</code>	Computes $(1 - y)$.
82		<code>fe_mul(ed_derivatives.one_minus_y_mul_x, ...)</code>	Computes $(1 - y) \cdot x$. <i>Precomputed for batch inversion.</i>

Located in fcmp_pp_crypto.cpp

8.4.2 Implementation Observations

There are no important notes or observations regarding this function.

8.4.3 Fuzzing Results

`point_to_ed_derivatives` is the entry point of `afl_wei_conv` and is also called by `output_tuple_to_pre_leaf_tuple`, so it is exercised by both the `wei_conv` (29,078 iters) and `output_to_tuple` (801,817 iters) campaigns. The explicit identity rejection on Line 1 prevents `fe_batch_invert` corruption: a single $1 - y = 0$ would zero every batched inversion via the `fe_invert(0) = 0` Fermat artifact noted in §4 (Batch Inverse). Across both harnesses the identity check fired before any inversion occurred whenever the input encoded $\mathcal{O}$, with zero ASan memory-safety findings.

8.5 Section Conclusion



The functions covered in this section all perform exactly as described mathematically, and are, in our opinion, fit for use.



9 Key Image

The purpose of this section is the following:

1. Verify the rationale that Monero's existing hash-to-point function is, in fact, biased.
2. Does the `unbiased_hash_to_ec` function achieve the stated goal of being unbiased?
3. Does the implementation match Elligator 2, specified in <https://eprint.iacr.org/2013/325.pdf>?
4. Is the algorithm utilized in the function unbiased from a cryptographic standpoint?
5. Is the algorithm utilized in the function collision resistant?
6. Does the code perform as claimed assuming the cryptographic protocol utilized in the function is unbiased?

9.1 Mathematical Overview

It should first be noted that the algorithm used is an application of an Elligator-style map, combined with an appropriate hash function. In the `unbiased_hash_to_ec`, the hash function used is `BLAKE2b`. Note that under the construction, the hash properties of the composite map are determined by the chosen hash function. It can be seen readily that the `unbiased_hash_to_ec` will inherit the cryptographic guarantees of `BLAKE2b`, and as a result will possess the same collision resistance as `BLAKE2b`.

The construction can be simply viewed as the Elligator 2 map φ with the `BLAKE2b` hash, H , giving:

$$\varphi(H(x))$$

Regardless of the properties of the outer map, the information is still protected by the hash on the inside.



9.2 PR #10338 - biased_hash_to_ec

9.2.1 Functionality

Listing 16: biased_hash_to_ec Code

```
1 static void biased_hash_to_ec(const public_key& key, ge_p3& res) {
2     hash h;
3     ge_p2 point;
4     ge_p1p1 point2;
5     cn_fast_hash(std::addressof(key), sizeof(public_key), h);
6     ge_fromfe_frombytes_vartime(&point, reinterpret_cast<const unsigned
7     char*>(&h));
8     ge_mul8(&point2, &point);
9     ge_p1p1_to_p3(&res, &point2);
10 }
```

Table 16: biased_hash_to_ec Source To Spec Map

L1	L2	Code	Purpose
616	5	cn_fast_hash(std::addressof(key), sizeof(public_key), h)	$h = \text{Keccak-256}(K)$, 32 bytes
617	6	ge_fromfe_frombytes_vartime(&point, ... &h)	$P = \ell_2(h \bmod p)$ in ge_p2
618	7	ge_mul8(&point2, &point)	$8 \cdot P$
619	8	ge_p1p1_to_p3(&res, &point2)	Convert to extended coordinates.

Located in crypto.cpp

9.2.2 Implementation Observations

1. The map is explicitly biased: `ge_fromfe_frombytes_vartime` is a Shallue–van de Woestijne-style construction whose image is restricted to points with square x -coordinate, so half of the prime-order subgroup is unreachable.
2. The cofactor is cleared by a single `ge_mul8` rather than by the RFC 9380 `clear_cofactor` pattern; for Ed25519 the two are equivalent, since the cofactor is exactly 8.
3. The hash is `cn_fast_hash` (Keccak-256) over a fixed-width 32-byte public key with no domain-separation tag. The fixed input width and Monero’s exclusive use of



`cn_fast_hash` for this construction make collisions across protocols implausible in current usage, but the same hardening recommendation given for `unbiased_hash_to_ec` (in the next subsection) applies if a second consumer of this primitive is added.

4. The function is declared `static`: it is file-internal to `crypto.cpp` and is invoked from the legacy stealth-address and ring-signature paths (`generate_key_image`, `generate_ring_signature`, `check_ring_signature`).

9.2.3 Fuzzing Results

`biased_hash_to_ec` is exercised on every iteration of `afl_hash_to_ec`. The campaign ran 100,015,388 iterations under ASan without crashes or ASan memory-safety findings, and the harness verified that every output decompresses as a valid Ed25519 point and is never the identity: so the bias of the map is a property of where outputs land, not whether outputs land on-curve.



9.3 PR #10338 - unbiased_hash_to_ec

9.3.1 Functionality

Listing 17: unbiased_hash_to_ec Code

```
1 void crypto_ops::unbiased_hash_to_ec(const unsigned char* preimage, const
  std::size_t length, ec_point& res) {
2   uint8_t hash[64];
3   blake2b(std::addressof(hash), 64, preimage, length, NULL, 0);
4
5   ge_p2 first;
6   ge_fromfe_frombytes_vartime(&first, reinterpret_cast<const unsigned
  char*>(&hash));
7   ge_p1p1 first_p1p1;
8   ge_mul8(&first_p1p1, &first);
9   ge_p3 first_p3;
10  ge_p1p1_to_p3(&first_p3, &first_p1p1);
11
12  ge_p2 second;
13  ge_fromfe_frombytes_vartime(&second, reinterpret_cast<const unsigned
  char*>(&hash) + 32);
14  ge_p1p1 second_p1p1;
15  ge_mul8(&second_p1p1, &second);
16  ge_p3 second_p3;
17  ge_p1p1_to_p3(&second_p3, &second_p1p1);
18  ge_cached second_cached;
19  ge_p3_to_cached(&second_cached, &second_p3);
20
21  ge_p1p1 point;
22  ge_add(&point, &first_p3, &second_cached);
23
24  ge_p3 res_ge_p3;
25  ge_p1p1_to_p3(&res_ge_p3, &point);
26  ge_p3_tobytes(&res, &res_ge_p3);
27 }
```




Table 17: unbiased_hash_to_ec Source To Spec Map

L1	L2	Code	Purpose
623	2	<code>uint8_t hash[64]</code>	Allocate for $(h1 h2)$
624	3	<code>blake2b(std::addressof(hash), 64, preimage,...)</code>	$(h1 h2) = \text{BLAKE2b}(\text{preimage})$
			64 bytes
627	6	<code>ge_fromfe_frombytes_vartime(&first, ... &hash)</code>	$P_1 = \ell_2(h1)$ from $hash[0..31]$
629	8	<code>ge_mul8(&first_p1p1, &first)</code>	$Q_1 = 8 \cdot P_1$
			(cofactor clear on first half)
631	10	<code>ge_p1p1_to_p3(&first_p3, &first_p1p1)</code>	Convert Q_1 to extended coordinates.
634	13	<code>ge_fromfe_frombytes_vartime(&second, ... &hash + 32)</code>	$P_2 = \ell_2(h2)$ from $hash[32..63]$
636	15	<code>ge_mul8(&second_p1p1, &second)</code>	$Q_2 = 8 \cdot P_2$
			(cofactor clear on second half)
638	17	<code>ge_p1p1_to_p3(&second_p3, &second_p1p1)</code>	Convert Q_2 to extended coordinates.
640	19	<code>ge_p3_to_cached(&second_cached, &second_p3)</code>	Convert Q_2 for addition
			<code>ge_cached</code> format
643	22	<code>ge_add(&point, &first_p3, &second_cached)</code>	$H = Q_1 + Q_2$
			(RFC 9380, §3: random-oracle
			construction $Q_0 + Q_1$)
646	25	<code>ge_p1p1_to_p3(&res_ge_p3, &point)</code>	Convert H to extended coordinates.
647	26	<code>ge_p3_tobytes(&res, &res_ge_p3)</code>	Compress H to 32 bytes.

Located in crypto.cpp

9.3.2 Proof of Unbiasedness

Let us assume first that Elligator 2 outputs only points with a square x -coordinate, which is only half of the points on the elliptic curve. The proposed solution by Kayaba was to invoke twice and sum the outputs to remove any bias. Here, we will confirm that this is the case.

First, we show that a sum of squares can represent any point on the curve, so let E be an elliptic curve of prime order p . Formally, we will show that for all $c \in \mathbb{F}_p$, we can find a, b such that:

$$a^2 + b^2 \equiv c \pmod{p}$$

Let S be the set of squares in $\mathbb{F}_p$. If we include 0, then by Euler's criterion:

$$|S| = \frac{p+1}{2}$$

Select an arbitrary c and define the set difference as $S_c = \{c - s \mid s \in S\}$. Since this is simply a translation, the cardinality is invariant; hence, it must be that $|S| = |S_c|$. We then note that:

$$|S| + |S_c| = p + 1 > p = |\mathbb{F}_p|$$

As such, the intersection of S and S_c is non-empty. Since we've established the existence of an $x \in S \cap S_c$, it must be both a square, and c minus a square: that is, there exist $a \in S$ and $b \in S_c$ such that

$$x = a^2 \quad \text{and} \quad x = c - b^2.$$

Hence, $a^2 + b^2 = c$.

Now we verify uniformity. We have shown that summing these coordinates can range the whole space, but it could be that certain points are much more likely outcomes than others, so we must establish that sums of this type are distributed uniformly. To show this, we count the number of solutions to $a^2 + b^2 \equiv c \pmod{p}$.

Let $p > 2$. Let χ be the Legendre symbol, defined as follows:

$$\chi(x) = \begin{cases} 0 & \text{if } x = 0, \\ 1 & \text{if } x \text{ is a quadratic residue,} \\ -1 & \text{if } x \text{ is a non-residue.} \end{cases}$$

For any $y \in \mathbb{F}_p$, the number of solutions to the equation $x^2 \equiv y \pmod{p}$ is exactly $1 + \chi(y)$. This can be readily seen by counting cases: if $y = 0$, there is only one solution, $x = 0$. If



y is a residue, then there exists a solution to $x^2 \equiv y \pmod{p}$, which means that $-x$ is also a solution, hence there must be two. If y is a non-residue, then by definition, there exists no solutions.

Let N_c be the number of solutions to $a^2 = c - b^2$. If we sum the number of ways to select a for every possible b , we find that

$$N_c = \sum_{x \in \mathbb{F}_p} (1 + \chi(x)) \cdot (1 + \chi(c - x)).$$

This comes from fixing a dummy variable x , then counting the number of a, b simultaneously satisfying $x = a^2$ and $b^2 = c - x$.

This summation can be expanded to obtain

$$N_c = \sum_{x \in \mathbb{F}_p} 1 + \sum_{x \in \mathbb{F}_p} \chi(x) + \sum_{x \in \mathbb{F}_p} \chi(c - x) + \sum_{x \in \mathbb{F}_p} \chi(x(c - x)).$$

Immediately, we note that the first summation is equal to p . We also notice that the number of residues is equal to the number of non-residues, so the second and third summation are both 0. From this, we simplify

$$N_c = p + \sum_{x \in \mathbb{F}_p} \chi(x(c - x)).$$

Since $\chi(x(c - x)) = 0$ when $x = 0$, we need only to sum over the non-zero values. If $x, c \neq 0$, by the multiplicativity of the character, we can factor $\chi(x(c - x))$ as

$$\chi(x(c - x)) = \chi(x^2(cx^{-1} - 1)) = \chi(x^2) \cdot \chi(cx^{-1} - 1).$$

Now, notice that since x iterates over all elements of $\mathbb{F}_p^*$, so does x^{-1} . Additionally, multiplication by c simply permutes the elements, so cx^{-1} iterates over all $\mathbb{F}_p^*$. Hence, $cx^{-1} - 1$ iterates over $\mathbb{F}_p \setminus \{-1\}$. Summing $\chi(y)$ over all non-zero y gives us 0, but this sum is over the whole space **except** -1 , the sum evaluates to $0 - \chi(-1) = -\chi(-1)$.

By Euler's criterion,

$$\chi(a) = a^{\frac{p-1}{2}} \pmod{p} \Rightarrow \chi(-1) = (-1)^{\frac{p-1}{2}}.$$

Overall, we have

$$N_c = p - (-1)^{\frac{p-1}{2}}.$$



If $c = 0$, then the summand is $\chi(-x^2) = \chi(-1) \cdot \chi(x^2)$. If $x = 0$, this term is 0, while if x is non-zero, then $\chi(x^2) = 1$ as it is guaranteed a residue. Applying Euler's criterion once more, the number of solutions for $c = 0$ is

$$N_0 = p + (p - 1)(-1)^{\frac{p-1}{2}}.$$

So, we have the number of solutions for an arbitrary c is given by

$$N_c = \begin{cases} p + (p - 1)(-1)^{\frac{p-1}{2}} & c = 0, \\ p - (-1)^{\frac{p-1}{2}} & c \neq 0. \end{cases}$$

Since the number of solutions to $a^2 + b^2 \equiv c \pmod{p}$ when $c \neq 0$ is bounded by $p + 1$, the probability of randomly selecting a, b and generating a specific c is bounded by

$$\frac{p + 1}{p^2} \approx \frac{1}{p},$$

so this process appears uniform. If $c = 0$, we expect slightly more bias: N_0 is bounded by $2p - 1$, thus

$$\frac{2p - 1}{p^2} \approx \frac{2}{p},$$

however in both cases this bias is negligible for large p .

When $p = 2^{255} - 19$, the prime that we are interested in, we have that $N_0 = 2p - 1$ and $N_c = p - 1$ for non-zero c , which yields the same conclusion as above: the bias introduced here is negligible. Therefore, we conclude that summing two Elligator outputs is sufficient to remove any bias.

9.3.3 Implementation Observations

1. The cofactor clearing happens separately before addition. In RFC 9380 Section 3, it is specified that this clearing happens after summation. While the result is mathematically equivalent, it should be noted that it does not exactly match the spec.
2. There is no domain-separating tag used when **BLAKE2b** is called, which is required by RFC 9380. The simple-API call `blake2b(out, 64, preimage, len, NULL, 0)` leaves **BLAKE2b**'s `personal` and `salt` parameters at their default zero, and the single in-tree call site (`derive_key_image_generator`) prepends no DST string to the preimage. Cross-protocol confusion is not exploitable today. The only caller hashes a 32-byte `public_key`, and every other Monero hash of a public key uses Keccak rather



than BLAKE2b, so the input domains are already separated by hash-function family. As a hardening recommendation before a second BLAKE2b-based hash-to-curve flavor enters the codebase, the `personal` field should be set to a unique 16-byte tag (e.g. "FCMP_KI_GEN_v1") using the `blake2b_init_param` API.

9.3.4 Edge Case Analysis

Input	Behavior
$O = \mathcal{O}$	Explicit check rejects.
$C = \mathcal{O}$	Explicit check rejects.
Legacy Output with Torsion	Torsion cleared, if 8 torsion, rejected.
Carrot Output	Torsion clearing skipped.
Point with $x = 0$	Legacy rejects when torsion clearing,
	Carrot passes identity check.

9.3.5 Fuzzing Results

The same AFL++ campaign `afl_hash_to_ec` exercised `unbiased_hash_to_ec` for 100,015,388 iterations with zero memory-safety findings and zero invariant violations: the harness checked both that the output decompresses as a valid Ed25519 point and that it is never the identity. The double-cofactor-clear and the cached/extended conversions all ran cleanly under ASan; the campaign therefore covers the order-of-operations deviation from RFC 9380 §3 noted in observation 1 and confirms it does not produce off-curve points or memory hazards.



9.4 PR #10338 - derive_key_image_generator

9.4.1 Functionality

Listing 18: derive_key_image_generator Code

```
1 void crypto_ops::derive_key_image_generator(const public_key& pub, const
    bool biased, ec_point& ki_gen) {
2     if (biased)
3         biased_derive_key_image_generator(pub, ki_gen);
4     else
5         unbiased_derive_key_image_generator(pub, ki_gen);
6 }
```

Table 18: derive_key_image_generator Source To Spec Map

L1	L2	Code	Purpose
660	1	void crypto_ops::derive_key_image_generator(...)	Function Definition
661	2	if (biased)	Use biased generator.
663	4	else	Use unbiased generator.

Located in crypto.cpp

9.4.2 Implementation Observations

1. It appears that this function simply acts as a switch in order to preserve legacy key images.



9.5 PR #10338 - generate_key_image

9.5.1 Functionality

Listing 19: generate_key_image Code

```
1 void crypto_ops::generate_key_image(const public_key& pub, const
    secret_key& sec, key_image& image) {
2     ge_p3 point;
3     ge_p2 point2;
4     assert(sc_check(&sec) == 0);
5     biased_hash_to_ec(pub, point);
6     ge_scalarmult(&point2, &unwrap(sec), &point);
7     ge_tobytes(&image, &point2);
8 }
```

Table 19: generate_key_image Source To Spec Map

L1	L2	Code	Purpose
671	4	assert(sc_check(&sec) == 0)	Validate secret key.
672	5	biased_hash_to_ec(pub, point)	$H_p = 8 \cdot \ell_2(\text{Keccak-256}(K))$
673	6	ge_scalarmult(&point2, &unwrap(sec), &point)	$I = x \cdot H_p$
674	7	ge_tobytes(&image, &point2)	Compress key image.

Located in crypto.cpp

9.5.2 Implementation Observations

1. This function always uses the biased hash to ec, but this is pre-FCMP++ and Carrot only.

9.5.3 Fuzzing Results

generate_key_image is fuzzed transitively through its biased_hash_to_ec dependency by afl_hash_to_ec (100,015,388 iters, zero findings). No dedicated harness was built for generate_key_image itself because it requires a valid secret scalar (asserted by sc_check), and a fuzzer over uniformly random 32-byte secrets would be rejected at the first line ~



half the time without exercising new behavior; the deterministic `ge_scalarmult` output is instead verified by GTest with RFC-derived known-answer vectors.

9.6 Section Conclusion



The functions described above are implemented in a way consistent with the underlying mathematics. As such, they appear fit for use.



10 Converting Outputs to Tuples

The purpose of this section is to address the following:

1. Verify that none of O, I, C can have torsion, and contain correct identity checks, both from a cryptographic standpoint and correctly implemented coding standpoint.
2. Verify the claim that no output or commitment detectable as a valid receive on the Monero blockchain today (or at any point in the past) would cause `output_to_tuple` to throw.

10.1 Mathematical Overview

The leaf consumed by the FCMP++ Merkle tree is the triple (O, I, C) , where O is the output public key, I is the key-image generator associated with O , and C is the Pedersen commitment to the amount. The functions in this section take a structurally heterogeneous `OutputPair` (Legacy or Carrot; see §10 for the dispatch surface) and assemble the canonical 96-byte (O, I, C) tuple on the prime-order subgroup of ED25519: torsion is cleared on Legacy inputs (using `get_valid_torsion_cleared_point` from §6); identity points are explicitly rejected so that a single $1 - y = 0$ cannot zero every batched inversion downstream (the `fe_invert(0) = 0` Fermat artifact discussed in §4); and I is derived from the original O rather than from its torsion-cleared counterpart, in order to preserve the 1-to-1 correspondence between key images and historical outputs.

10.2 PR #10360 - `output_to_tuple`

10.2.1 Functionality

`output_to_tuple` is the consumer of the five PR #10358 dispatch helpers documented in the previous section (`output_pubkey_cref`, `commitment_cref`, `output_checked_for_torsion`, `use_biased_hash_to_point`, and `output_tuple_from_bytes`). Although the function itself is introduced as part of the same upstream commit family as the type infrastructure of PR #10358, its analysis is most natural here, alongside `output_tuple_to_pre_leaf_tuple`, since both functions belong to the leaf-construction pipeline.

Listing 20: `output_to_tuple` Code

```
1 OutputTuple output_to_tuple(const OutputPair& output_pair)
2 {
3     const crypto::public_key& output_pubkey = output_pubkey_cref(
        output_pair);
```



```
4     const crypto::ec_point& commitment = commitment_cref(output_pair);
5
6     crypto::ec_point O = output_pubkey;
7     crypto::ec_point C = commitment;
8
9     // If the output has already been checked for torsion, then we don't
10    need to clear torsion here
11    if (!output_checked_for_torsion(output_pair))
12    {
13        TIME_MEASURE_NS_START(clear_torsion_ns);
14
15        if (!fcmp_pp::get_valid_torsion_cleared_point(output_pubkey, O))
16            throw std::runtime_error("O is invalid for insertion to tree");
17
18        if (!fcmp_pp::get_valid_torsion_cleared_point(commitment, C))
19            throw std::runtime_error("C is invalid for insertion to tree");
20
21        if (O != output_pubkey)
22            LOG_PRINT_L2("Output pubkey has torsion: " << output_pubkey);
23        if (C != commitment)
24            LOG_PRINT_L2("Commitment has torsion: " << commitment);
25
26        TIME_MEASURE_NS_FINISH(clear_torsion_ns);
27
28        LOG_PRINT_L3("clear_torsion_ns: " << clear_torsion_ns);
29    }
30
31    #if !defined(NDEBUG)
32    {
33        // Debug build safety checks
34        crypto::ec_point O_debug;
35        crypto::ec_point C_debug;
36        assert(fcmp_pp::get_valid_torsion_cleared_point(output_pubkey,
37        O_debug));
38        assert(fcmp_pp::get_valid_torsion_cleared_point(commitment,
39        C_debug));
40        assert(O == O_debug);
41        assert(C == C_debug);
42    }
43
44    #endif
45
46    // Redundant check for safety
```



```

42     if (0 == crypto::EC_I)
43         throw std::runtime_error("0 cannot equal identity");
44     if (C == crypto::EC_I)
45         throw std::runtime_error("C cannot equal identity");
46
47     TIME_MEASURE_NS_START(derive_key_image_generator_ns);
48
49     // Derive key image generator using original output pubkey
50     crypto::ec_point I;
51     crypto::derive_key_image_generator(output_pubkey,
52     use_biased_hash_to_point(output_pair), I);
53
54     TIME_MEASURE_NS_FINISH(derive_key_image_generator_ns);
55
56     LOG_PRINT_L3("derive_key_image_generator_ns: " <<
57     derive_key_image_generator_ns);
58
59     return output_tuple_from_bytes(0, I, C);
60 }
61 }

```

Table 20: output_to_tuple Source To Spec Map

L1	L2	Code	Purpose
62	3	output_pubkey_cref(output_pair)	Extract O_{raw} from OutputPair.
63	4	commitment_cref(output_pair)	Extract C_{raw} from OutputPair.
68	9	if (!output_checked_for_torsion(output_pair))	Legacy outputs need torsion clearing.
			Carrot outputs skip.
72	13	get_valid_torsion_cleared_point(output_pubkey, 0)	Clear torsion on O_{raw} .
73	14	get_valid_torsion_cleared_point(commitment, C)	Clear torsion on C_{raw} .
90	31	if (0 == crypto::EC_I) throw	Reject identity after torsion clearing.
98	39	derive_key_image_generator(...)	$I = \text{HashToPoint}(O_{raw})$.
102	43	return output_tuple_from_bytes(0, I, C)	Serialize (O, I, C) into OutputTuple

Located in curve_trees.cpp



10.2.2 Implementation Observations

1. Note that in Line 39 of the included code, the value included in the `HashToPoint` argument is the original pubkey, not the torsion cleared on computed. This is due to the torsion clearing process collapsing two possible distinct keys to the same cleared one. For example, for $T_1 \neq T_2$ torsion elements, $O_1 = P_0 + T_1$ and $O_2 = P_0 + T_2$ implies that $O_1 \neq O_2$ however, after torsion clearing, they map to the same value.

10.2.3 Edge Case Analysis

Input	Behavior
$O = \mathcal{O}$	Explicit check rejects.
$C = \mathcal{O}$	Explicit check rejects.
Legacy Output with Torsion	Torsion cleared, if 8 torsion, rejected.
Carrot Output	Torsion clearing skipped.
Point with $x = 0$	Legacy rejects when torsion clearing,
	Carrot passes identity check.

10.2.4 Fuzzing Results

The AFL++ harness `afl_output_to_tuple` exercised the full pipeline (`output_to_tuple` $\rightarrow$ `output_tuple_to_pre_leaf_tuple`) for 801,817 iterations across ASan (308,443), CMPLOG (402,193), and extended-ASan (91,181) rounds. The CMPLOG and core ASan rounds reported zero crashes; the 96-byte input format ($O_{raw} || C_{raw} || \text{flags}$) confirmed that the explicit identity checks reject $\mathcal{O}$ for both O and C . The extended-ASan round recorded 12 SIGILL assertion violations, all of the same non-canonical-encoding class observed in `afl_torsion/afl_wei_conv` and triaged as input-precondition violations rather than memory-safety findings.



10.3 PR #10360 - output_tuple_to_pre_leaf_tuple

10.3.1 Functionality

Listing 21: output_tuple_to_pre_leaf_tuple Code

```
1 static PreLeafTuple output_tuple_to_pre_leaf_tuple(const OutputTuple& o)
2 {
3     TIME_MEASURE_NS_START(point_to_ed_derivatives_ns);
4
5     const crypto::ec_point& O = (crypto::ec_point&)o.O;
6     const crypto::ec_point& I = (crypto::ec_point&)o.I;
7     const crypto::ec_point& C = (crypto::ec_point&)o.C;
8
9     // TODO: we end up decompressing O and C twice, both in here and when
10    // checking the points for torsion. It's worth
11    // checking how much of an impact that has on performance.
12    PreLeafTuple plt;
13    if (!fcmp_pp::point_to_ed_derivatives(O, plt.O_derivatives))
14        throw std::runtime_error("failed to get ed derivatives from O");
15    if (!fcmp_pp::point_to_ed_derivatives(I, plt.I_derivatives))
16        throw std::runtime_error("failed to get ed derivatives from I");
17    if (!fcmp_pp::point_to_ed_derivatives(C, plt.C_derivatives))
18        throw std::runtime_error("failed to get ed derivatives from C");
19
20    TIME_MEASURE_NS_FINISH(point_to_ed_derivatives_ns);
21
22    LOG_PRINT_L3("point_to_ed_derivatives_ns: " <<
23    point_to_ed_derivatives_ns);
24
25    return plt;
26 }
```



Table 21: output_to_tuple Source To Spec Map

L1	L2	Code	Purpose
126	5	(crypto::ec_point&) o.O	Reinterpret 32-byte OutputTuple.O
			as compressed ED25519 point.
132	11	point_to_ed_derivatives(O, plt.O_derivatives)	$O \rightarrow (1 + y_O, 1 - y_O, (1 - y_O) \cdot x_O)$
134	13	point_to_ed_derivatives(I, plt.I_derivatives)	$I \rightarrow (1 + y_I, 1 - y_I, (1 - y_I) \cdot x_I)$
136	15	point_to_ed_derivatives(C, plt.C_derivatives)	$C \rightarrow (1 + y_C, 1 - y_C, (1 - y_C) \cdot x_C)$

Located in curve_trees.cpp

10.3.2 Implementation Observations

1. Each `point_to_ed_derivatives` call decompresses the 32-byte point, rejects the identity and computes the three field elements. The inversions are deferred to the batch step.
2. On the legacy (non-Carrot) output branch, O and C are decompressed twice per output: once during torsion clearing inside `output_to_tuple` (via `get_valid_torsion_cleared_point`, which calls `ge_frombytes_vartime` and then re-encodes the cleared point with `ge_p3_tobytes`), and again here in `point_to_ed_derivatives`. The source itself acknowledges this with a `TODO` at `curve_trees.cpp:128`. The cost is performance only, with no correctness or security impact, but at scale (full-chain re-sync) the redundant decompressions accumulate.

10.3.3 Fuzzing Results

Inherits the `afl_output_to_tuple` campaign coverage above (801,817 iterations across ASan, CMPLOG, and extended-ASan, zero memory-safety findings). The three `point_to_ed_derivatives` calls are the same routine fuzzed independently in §8 above; their identity-rejection branches were exercised on every O -valued input the corpus produced.



10.4 Section Conclusion



It can be seen readily that the `output_to_tuple` function explicitly checks for torsion in the (O, I, C) points, as well as checking to make sure none of the points are the identity.



11 Constructing the Selene Scalar

The purpose of this section is as follows:

1. Verify that `SeleneScalar` is the correct C representation of the corresponding Rust Object.
2. Verify that `fcmp_pp::tower_cycle::selene_scalar_from_bytes` calls `selene_scalar_from_bytes` in `fcmp.h` safely from a runtime and memory standpoint.
3. Verify that the `selene_scalar_from_bytes` in `lib.rs` is implemented correctly from a runtime and memory standpoint and calls `read_F` in a safe manner from the same standpoint, as well as a cryptographically secure point.
4. Verify that the Rust FFI layer is set up in a manner that is fit for production use in Monero.

11.1 PR #10359 - `selene_scalar_from_bytes`

11.1.1 Functionality

Listing 22: `selene_scalar_from_bytes` Code

```
1 SeleneScalar selene_scalar_from_bytes(const crypto::ec_coord& bytes)
2 {
3     SeleneScalar selene_scalar;
4     int r = ::selene_scalar_from_bytes(to_bytes(bytes), &selene_scalar);
5     CHECK_FFI_RES;
6     return selene_scalar;
7 }
```

11.1.2 Implementation Observations

1. The C++ wrapper is purely a marshalling layer: it stack-allocates a `SeleneScalar`, calls the Rust FFI `::selene_scalar_from_bytes` on the raw `const uint8_t*` produced by `to_bytes(bytes)`, enforces `r == 0` via `CHECK_FFI_RES`, and returns by value with NRVO. Non-zero return codes propagate as a Monero exception via `CHECK_AND_ASSERT_THROW_MES` rather than corrupting the output.
2. The C declaration `struct SeleneScalar { uintptr_t _0[32 / sizeof(uintptr_t)]; }` in `fcmp+.h` is hand-maintained (the file is cp'd into place by the build system, not re-generated by `cbindgen`). There is no `static_assert(sizeof(SeleneScalar) == 32)`



to catch ABI drift, even though the sibling output-pair types in `fcmp_pp_types.h` do carry such guards. ABI consistency relies further on `Cargo.toml` patching `crypto-bigint` to a non-default `c-repr` branch; the contract is correct today but unprotected against future drift. Adding a one-line size assertion at the C++ declaration would close the gap at zero cost.

3. Per audit scope, the Rust implementation of `::selene_scalar_from_bytes` in the `helioselene` crate is out of scope for this analysis; this audit verifies only the C-side FFI contract.

11.1.3 Edge Case Analysis

Input	Behavior
All-zero bytes	Rust returns 0; <code>SeleneScalar</code> holds canonical zero.
All-0xFF bytes (encodes $2^{256} - 1 \gg q$)	Rust's <code>Ciphersuite::read_F</code> rejects non-canonical encodings, returning the parse-error code <code>-2</code> ; <code>CHECK_FFI_RES</code> then throws.
Non-canonical $a \geq q$	Same behaviour as above: <code>read_F</code> rejects rather than reducing.
Rust returns non-zero <code>r</code>	<code>CHECK_FFI_RES</code> throws; output discarded.

11.1.4 Fuzzing Results

The libFuzzer harness `fuzz_selene_ffi` exercised the C/Rust FFI boundary for 1,500,000 iterations under ASan with 0 crashes, and 0 round-trip violations. The companion harness `fuzz_helios_ffi` ran an identical 1,500,000-iteration campaign on the sibling FFI with the same clean result. The harness exercises the deserialise-then-serialise symmetry through the `CHECK_FFI_RES` error path: any non-zero return from the Rust side (null-pointer code `-1`, parse-error code `-2`) propagates as a C++ exception via the macro, rather than corrupting the `SeleneScalar` output. Across 3,000,000 total iterations at the FFI boundary the campaign produced no memory hazards and no inconsistencies between the C++ and Rust sides.

11.1.5 Conclusion



The C wrapper correctly marshals input bytes, propagates Rust-side errors as exceptions, and returns the populated `SeleneScalar` with copy elision. Cryptographic correctness of



the conversion itself is delegated to a separate audit of the Rust crate; in our opinion, the C-side FFI is fit for use, with the caveat that a defense-in-depth `static_assert` on `sizeof(SeleneScalar)` should be added before this code reaches consensus-hot paths.



12 ED25519 to Curve Tree

The purpose of this section is to verify that `output_to_leaves` is implemented correctly from a cryptographic, memory and runtime standpoint.

12.1 PR #10360 - `output_to_leaves`

12.1.1 Functionality

Listing 23: `output_to_leaves` Code

```
1 void CurveTrees<C1, C2>::outputs_to_leaves(std::vector<UnifiedOutput>&&
    new_outputs,
2     std::vector<typename C1::Scalar>& flattened_leaves_out,
3     std::vector<UnifiedOutput>& valid_outputs_out)
4 {
5     flattened_leaves_out.clear();
6     valid_outputs_out.clear();
7
8     TIME_MEASURE_START(set_valid_leaves);
9
10    // Keep track of valid outputs to make sure we only use leaves from
    valid outputs. Can't use std::vector<bool>
11    // because std::vector<bool> concurrent access is not thread safe.
12    enum Boolean : uint8_t {
13        False = 0,
14        True = 1,
15    };
16    std::vector<Boolean> valid_outputs(new_outputs.size(), False);
17
18    tools::threadpool& tpool = tools::threadpool::getInstanceForCompute();
19    tools::threadpool::waiter waiter(tpool);
20    const std::size_t n_threads = std::max<std::size_t>(1, tpool.
    get_max_concurrency());
21
22    TIME_MEASURE_START(convert_valid_leaves);
23    // Step 1. Multithreaded convert valid outputs into Edwards
    derivatives needed to get Wei coordinates
24    std::vector<PreLeafTuple> pre_leaves;
25    pre_leaves.resize(new_outputs.size());
26    const std::size_t LEAF_CONVERT_BATCH_SIZE = std::max<std::size_t>(1, (
    new_outputs.size() / n_threads));
```



```
27     for (std::size_t i = 0; i < new_outputs.size(); i +=
LEAF_CONVERT_BATCH_SIZE)
28     {
29         const std::size_t end = std::min(i + LEAF_CONVERT_BATCH_SIZE,
new_outputs.size());
30         tpool.submit(&waiter,
31             [
32                 &new_outputs,
33                 &valid_outputs,
34                 &pre_leaves,
35                 i,
36                 end
37             ]()
38             {
39                 for (std::size_t j = i; j < end; ++j)
40                 {
41                     CHECK_AND_ASSERT_THROW_MES(!valid_outputs.at(j), "
unexpected already valid output");
42
43                     const auto& output_pair = new_outputs.at(j).
output_pair;
44                     try
45                     {
46                         pre_leaves.at(j) = output_to_pre_leaf_tuple(
output_pair);
47                     }
48                     catch (...)
49                     {
50                         /* Invalid outputs can't be added to the tree */
51                         LOG_PRINT_L2("Output " << new_outputs.at(j).
unified_id << " is invalid (out pubkey "
52                             << output_pubkey_cref(output_pair)
53                             << " , commitment " << commitment_cref(
output_pair) << ")");
54                         continue;
55                     }
56
57                     valid_outputs.at(j) = True;
58                 }
59             },
60             true
61         );
62     }
```



```
63
64     CHECK_AND_ASSERT_THROW_MES(waiter.wait(), "failed to convert outputs
to ed derivatives");
65     TIME_MEASURE_FINISH(convert_valid_leaves);
66
67     TIME_MEASURE_START(collect_derivatives);
68     // Step 2. Collect valid Edwards y derivatives
69     const std::size_t n_valid_outputs = std::count(valid_outputs.begin(),
valid_outputs.end(), True);
70     const std::size_t n_valid_leaf_points = n_valid_outputs *
LEAF_TUPLE_POINTS;
71
72     // Collecting [(1+y),(1-y),((1-y)*x)] for batch inversion
73     std::unique_ptr<fe[]> one_plus_y_vec = std::make_unique<fe[]>(
n_valid_leaf_points);
74     std::unique_ptr<fe[]> fe_batch = std::make_unique<fe[]>(
n_valid_leaf_points * 2);
75     std::unique_ptr<fe[]> batch_inv_res = std::make_unique<fe[]>(
n_valid_leaf_points * 2);
76
77     std::size_t valid_i = 0, batch_i = 0;
78     for (std::size_t i = 0; i < valid_outputs.size(); ++i)
79     {
80         if (!valid_outputs[i])
81             continue;
82
83         CHECK_AND_ASSERT_THROW_MES(n_valid_leaf_points > valid_i, "
unexpected valid_i");
84
85         auto& pl = pre_leaves.at(i);
86
87         auto& O_derivatives = pl.O_derivatives;
88         auto& I_derivatives = pl.I_derivatives;
89         auto& C_derivatives = pl.C_derivatives;
90
91         static_assert(LEAF_TUPLE_POINTS == 3, "unexpected n leaf tuple
points");
92
93         // TODO: avoid copying underlying (tried using pointer to pointers
, but wasn't clean)
94         memcpy(&one_plus_y_vec[valid_i++], &O_derivatives.one_plus_y,
sizeof(fe));
```



```
95     memcpy(&one_plus_y_vec[valid_i++], &I_derivatives.one_plus_y,
96     sizeof(fe));
97     memcpy(&one_plus_y_vec[valid_i++], &C_derivatives.one_plus_y,
98     sizeof(fe));
99     memcpy(&fe_batch[batch_i++], &O_derivatives.one_minus_y, sizeof(fe
100 ));
101     memcpy(&fe_batch[batch_i++], &O_derivatives.one_minus_y_mul_x,
102     sizeof(fe));
103     memcpy(&fe_batch[batch_i++], &I_derivatives.one_minus_y, sizeof(fe
104 ));
105     memcpy(&fe_batch[batch_i++], &I_derivatives.one_minus_y_mul_x,
106     sizeof(fe));
107     memcpy(&fe_batch[batch_i++], &C_derivatives.one_minus_y, sizeof(fe
108 ));
109     memcpy(&fe_batch[batch_i++], &C_derivatives.one_minus_y_mul_x,
110     sizeof(fe));
111 }
112
113 CHECK_AND_ASSERT_THROW_MES(n_valid_leaf_points == valid_i, "unexpected
114 end valid_i");
115 CHECK_AND_ASSERT_THROW_MES((n_valid_leaf_points * 2) == batch_i, "
116 unexpected end batch_i");
117 TIME_MEASURE_FINISH(collect_derivatives);
118
119 TIME_MEASURE_START(batch_invert);
120 // Step 3. Get batch inverse of all valid (1-y)'s and ((1-y)*x)'s
121 // - Batch inversion is significantly faster than inverting 1 at a
122 time
123 fe_batch_invert(batch_inv_res.get(), fe_batch.get(),
124 n_valid_leaf_points * 2);
125 TIME_MEASURE_FINISH(batch_invert);
126
127 TIME_MEASURE_START(get_selene_scalars);
128 // Step 4. Multithreaded get Wei coordinates and convert to Selene
129 scalars
130 const std::size_t n_valid_leaf_elems = n_valid_outputs *
131 LEAF_TUPLE_SIZE;
132 flattened_leaves_out.resize(n_valid_leaf_elems);
133 CHECK_AND_ASSERT_THROW_MES(flattened_leaves_out.size() == (2 *
134 n_valid_leaf_points),
```



```
123         "unexpected size of flattened leaves");
124
125     const std::size_t DERIVATION_BATCH_SIZE = std::max<std::size_t>(1, (
n_valid_leaf_points / n_threads));
126     for (std::size_t i = 0; i < n_valid_leaf_points; i +=
DERIVATION_BATCH_SIZE)
127     {
128         const std::size_t end = std::min(n_valid_leaf_points, i +
DERIVATION_BATCH_SIZE);
129         tpool.submit(&waiter,
130             [
131                 &batch_inv_res,
132                 &one_plus_y_vec,
133                 &flattened_leaves_out,
134                 i,
135                 end
136             ]()
137             {
138                 std::size_t point_idx = i * 2;
139                 for (std::size_t j = i; j < end; ++j)
140                 {
141                     crypto::ec_coord wei_x;
142                     crypto::ec_coord wei_y;
143                     fe_ed_derivatives_to_wei_x_y(
144                         to_bytes(wei_x),
145                         to_bytes(wei_y),
146                         batch_inv_res[point_idx]/*inv_one_minus_y*/,
147                         one_plus_y_vec[j],
148                         batch_inv_res[point_idx + 1]/*
inv_one_minus_y_mul_x*/
149                     );
150
151                     flattened_leaves_out[point_idx++] = tower_cycle::
selene_scalar_from_bytes(wei_x);
152                     flattened_leaves_out[point_idx++] = tower_cycle::
selene_scalar_from_bytes(wei_y);
153                 }
154             },
155             true
156         );
157     }
158
```



```
159     CHECK_AND_ASSERT_THROW_MES(waiter.wait(), "failed to convert outputs
to wei coords");
160     TIME_MEASURE_FINISH(get_selene_scalars);
161
162     // Step 5. Set valid tuples to be stored in the db
163     valid_outputs_out.reserve(n_valid_outputs);
164     for (std::size_t i = 0; i < valid_outputs.size(); ++i)
165     {
166         if (!valid_outputs[i])
167             continue;
168
169         // We can derive leaf tuples from output pairs, so we store just
the unified output in the db to save 32 bytes
170         valid_outputs_out.emplace_back(std::move(new_outputs.at(i)));
171     }
172     CHECK_AND_ASSERT_THROW_MES(valid_outputs_out.size() == n_valid_outputs
, "unexpected size of valid_outputs_out");
```




Table 22: output_to_leaves Source To Spec Map

L1	L2	Code	Purpose
179	25	<code>pre_leaves.resize(new_outputs.size())</code>	Allocate N PreLeafTuple slots
180	26	<code>LEAF_CONVERT_BATCH_SIZE=max(1, $N/n_{threads}$)</code>	Divide work across threadpool
200	46	<code>pre_leaves.at(j) = ...</code>	$(O_{raw}, C_{raw}) \Rightarrow \text{PreLeafTuple}$
202	48	<code>catch(...)</code>	Invalid inputs silently skipped.
211	57	<code>valid_outputs.at(j) = True</code>	Mark valid outputs for later filtering.
227	73	<code>one_plus_y_vec = make_unique<fe[]>(3N_v)</code>	Array of $1+y$ values.
228	74	<code>fe_batch = make_unique<fe[]>(6N_v)</code>	Array for batch inversion inputs.
248	94	<code>memcpy one_plus_y_vec for O, I, C</code>	Collect $(1+y)$ for each point.
252	98	<code>memcpy fe_batch for O, I, C</code>	Collect $(1-y), (1-y)x$ for each point.
269	115	<code>fe_batch_invert(batch_inv_res, fe_batch, 6N_v)</code>	Batch Inversion.
279	125	<code>DERIVATION_BATCH_SIZE = max(1, $3N_v/n_{threads}$)</code>	Divide $3N_v$ points across threads.
292	138	<code>point_idx = i * 2</code>	Index into <code>batch_inv_res</code> and
			<code>flattened_leaves_out</code> .
297	143	<code>fe_ed_derivatives_to_wei_x_y(...)</code>	Compute wei_x, wei_y per PR 10345.
305	151	<code>selene_scalar_from_bytes(wei_x)</code>	Converts 32 byte wei_x to SeleneScalar
			via Rust FFI.
306	152	<code>selene_scalar_from_bytes(wei_y)</code>	Converts 32 byte wei_y to SeleneScalar
			via Rust FFI.
320	166	<code>if (!valid_outputs[i]) continue</code>	Skip invalid outputs.
324	170	<code>valid_outputs_out.emplace_back(...)</code>	Collect valid UnifiedOutput entries.

Located in curve_trees.cpp



12.1.2 Implementation Observations

1. `output_to_leaves` is the public composition of `output_to_tuple` and `output_tuple_to_pre_leaf_tuple`, followed by batch inversion (`fe_batch_invert`), Weierstrass conversion (`fe_ed_derivatives_to_wei_x_y`), and FFI marshalling (`selene_scalar_from_bytes`). The output is a flattened vector of 6 `SeleneScalars` per output (three points $\times$ two Weierstrass coordinates).
2. Per-output exceptions thrown during torsion clearing or identity rejection are caught by the threadpool lambda and the corresponding output is recorded as invalid, preserving forward progress on the rest of the batch.
3. Identity rejection inside `point_to_ed_derivatives` is important for correctness since a $1 - y = 0$ entry would propagate through Montgomery's trick in `fe_batch_invert` and zero out *every* inversion in the batch, not just the offending one.

12.1.3 Edge Case Analysis

Input	Behavior
Empty batch ($N = 0$)	No-op; returns empty leaf vector.
O or $C = \mathcal{O}$ in any output	<code>output_to_tuple</code> throws; output marked invalid.
Legacy output with 8-torsion	Rejected during torsion clearing.
Carrot output	Torsion clearing skipped; identity check still applied.
$y_P = 1$ for any $P \in \{O, I, C\}$	Identity rejection in <code>point_to_ed_derivatives</code> throws; batch-inverse poisoning prevented.

12.1.4 Fuzzing Results

`outputs_to_leaves` is the public composition of `output_to_tuple` and `output_tuple_to_pre_leaf_tuple`; it inherits the `afl_output_to_tuple` campaign coverage from §11 above (801,817 iterations across ASan, CMPLOG, and extended-ASan rounds, zero memory-safety findings). A GTest end-to-end pipeline test (`test_e2e_pipeline.cpp`) exercises the full leaf-construction path on representative legacy and Carrot output vectors and confirms the expected leaf encoding on each.



12.1.5 Conclusion



The pipeline correctly composes torsion clearing, identity rejection, batch inversion, Weierstrass conversion, and FFI scalar marshalling, with per-output exception isolation that prevents a single malformed output from corrupting the rest of the batch. In our opinion, the implementation is fit for use.



13 Conclusion

It is our opinion that the functions explored in this paper are nearly entirely fit for use and for deployment in Monero. Each of the audit goals have been addressed as written and answered in the affirmative in almost all cases.

The code does a wonderful job of accurately implementing the sound mathematical structure necessary and achieves exactly what it is described as doing.